



数学建模校内竞赛论文



论文题目：共享停车位调配的优化建模与在线调度策略

组号：12772

成员：贾泽昊 毕艳泽 史润

选题：B

姓名	学院	年级	专业	学号	联系电话	高等数学	线性代数	概率统计	CET4	CET6
贾泽昊	电气工程学院	2024	电气工程及其自动化	20241211	18503177605	93	87	83	596	540
毕艳泽	电气工程学院	2024	电气工程及其自动化	20241237	17823419894	88	87	92	576	561
史润	电气工程学院	2024	电气工程及其自动化	20241220	13098790065	97	95	95	462	

日期

共享停车位资源调配的优化建模与在线调度策略

摘要

本文以智慧城市共享停车位资源调配问题为研究对象，基于 300 个共享车位与 500 位用户请求的真实数据，建立从静态匹配、时空约束分配到在线调度和供需分析的层层递进模型体系，对共享停车资源配置问题进行了系统研究。

针对第一问，构建基于距离可达性的**二分图匹配模型**，并采用两阶段整数规划实现服务规模与步行距离的字典序优化。结果表明，在忽略时间约束条件下，可实现全部 **300 个车位满负荷利用**，平均步行距离仅为 **2.82 m**。

针对第二问，在问题一基础上引入时间覆盖约束与时间冲突互斥约束，建立**时空约束下的混合整数线性规划模型**。通过构造时间兼容边集和冲突集合，实现共享车位的时空协同调度。结果表明服务用户数**下降至 220 位**，表明时间兼容性是制约系统服务能力的主要因素。

针对第三问，针对用户请求实时到达且未来信息未知的特点，构建**在线资源分配模型**，并设计距离优先与负载均衡两类在线贪心策略。结果表明，Greedy-Nearest 策略达到**离线最优解 90.9%** 的服务水平，能够满足实时调度需求，在线损失主要来源于高峰期资源竞争。

针对第四问，建立供需压力评价模型，从时间和空间两个维度分析共享停车资源的供需匹配状况。研究发现**午间时段供需矛盾**最为突出，部分区域存在持续性供给盲区；灵敏度分析表明，扩展车位开放时段能够显著提升系统服务能力。

关键词：共享停车；整数线性规划；混合整数线性规划；在线调度算法；贪心策略；供需分析

一、问题重述

问题背景

在城市核心功能区内，不同停车资源在时间维度上存在明显闲置现象。部分停车位仅在特定时段被使用，而周边用户在工作、办事等活动过程中产生阶段性停车需求。为提高停车资源利用效率，管理平台收集停车位开放信息与用户停车申请信息，并根据空间位置和时间需求进行统一调度。

问题提出

现有一组共享停车数据：区域内分布 300 个共享车位，每个车位有确定的空间坐标和可用时段集合；500 位用户提交了停车请求，每条请求包含目的地坐标、最大步行距离及计划到达和离开时间。要求依次解决以下四个问题：

问题一：所有车位全天可用，在同一时刻为用户分配车位，每位用户至多分配一个车位、每个车位至多服务一位用户。目标为最大化服务用户数，再最小化用户步行距离之和。

问题二：车位仅在特定时段可用，用户有到达和离开时间约束。需在时间兼容前提下分配车位，尽可能扩大服务规模并缩短步行距离。

问题三：用户请求按到达时间依次出现，平台须立即决策是否分配且不可更改。需设计在线调度策略并与离线最优解对比。

问题四：综合以上建模与计算，分析共享停车资源的供需匹配状况，为平台运营提供优化方向。

二、问题分析

问题一的分析

用户与车位之间存在一对一分配关系，且用户仅能选择步行距离不超过允许范围的车位，因此首先需要确定用户与车位之间的可行匹配关系。在满足车位容量约束和用户

唯一分配约束的条件下，优先提高服务用户数量；当服务规模达到最大后，再进一步优化用户步行距离。因此可将问题构建为二分图匹配下的双层优化问题 [5]。

问题二的分析

引入时间约束后，模型需要同时考虑空间可达性与时间兼容性。同一车位在时间不重叠条件下可服务多个用户，因此车位具有时间复用特征。资源竞争从空间竞争扩展为时空双重竞争，决策需同时选择服务哪些用户并协调用户与车位之间的时间分配关系。这一变化使问题转化为带时间窗的资源约束分配问题，需引入时间冲突约束来刻画车位内用户之间的互斥关系。

问题三的分析

问题三要求平台按照用户请求到达顺序实时完成车位分配，并在当前时刻立即给出接受或拒绝决策。与问题二相比，平台无法提前获得未来用户请求信息，因此原有基于全局信息的混合整数规划模型无法直接应用。

在线场景下，系统状态随用户到达不断变化。某一时刻对当前用户作出的分配决策不仅影响当前服务质量，同时会改变后续时段可利用资源结构，因此平台面临典型的在线资源分配问题。

进一步观察可知，车位资源具有明显的时间占用特征：用户一旦被接受，其对应时间区间内的资源容量将被锁定，而未来需求尚未知晓。因此平台需要在”立即满足当前需求”与”保留资源应对未来需求”之间进行权衡。

由于在线决策要求毫秒级响应，难以采用复杂优化算法实时求解，因此本文基于局部最优思想构建在线贪心调度模型，并分别从距离优先和资源均衡两个角度设计在线决策策略，以离线最优解作为性能基准评价在线损失 [2, 9]。

问题四的分析

问题四需从时间和空间两个维度评估供需匹配特征。时间维度上，离散化时间轴统计各时刻的活跃需求与可用供给；空间维度上，将区域划分为网格并计算各网格的可达

供给。结合灵敏度分析，为平台运营提供参考依据。

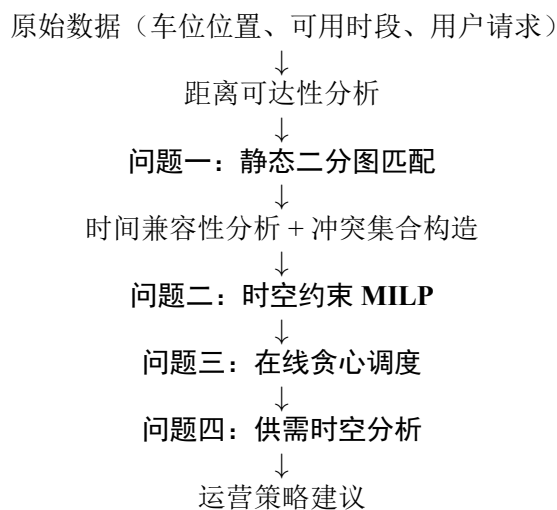


图 1: 全文建模框架

三、模型假设

- 平面几何假设：**所有车位和目的地所在区域 $[0, 100] \times [0, 100]$ 为理想欧几里得平面，用户沿直线步行，忽略实际道路障碍。坐标数据来源于标准化平面坐标，单位并非真实地理米数，因此文中所涉及的距离数值均表示相对尺度。
- 时间准确性假设：**假设用户提交的到达时间和离开时间准确可靠，不考虑因交通拥堵等因素导致的时间偏移。
- 资源同质性假设：**忽略车位大小、车型匹配等物理差异，认为所有车位对于任意车辆均等价可用。
- 静态瞬时假设：**对于问题一，假设所有用户的停车请求在空间上同时存在，忽略到达与离开的时序先后对车位占用的影响。
- 完全履约假设：**假设用户一旦被分配车位即按时到达并使用，不考虑用户取消或未按时到达的情况。

四、符号说明

符号	含义
S	共享车位集合, $S = \{1, 2, \dots, 300\}$
U	用户请求集合, $U = \{1, 2, \dots, 500\}$
$(x_i^{\text{spot}}, y_i^{\text{spot}})$	车位 i 的二维空间坐标
$(x_j^{\text{dest}}, y_j^{\text{dest}})$	用户 j 的目的地空间坐标
d_{ji}	车位 i 与用户 j 目的地之间的欧氏距离
w_j	用户 j 可承受的最大步行距离
$[a_j, d_j]$	用户 j 的停车时段, a_j 为到达时间、 d_j 为离开时间
$T_i = \{[s_i^{(k)}, e_i^{(k)}]\}$	车位 i 的可用时段集合
x_{ji}	0-1 决策变量, 表示是否将车位 i 分配给用户 j
N^*	最大可服务用户数
E, E^T	距离可行边集、时空可行边集
C_i	车位 i 内的用户时间冲突对集合

五、模型的建立与求解

5.1 问题一：静态分配模型

每个用户至多分配一个车位, 每个车位至多服务一位用户, 因此用户与车位之间形成一对一匹配关系。用户仅会接受步行距离不超过自身阈值的车位, 据此仅保留满足距离约束的用户-车位组合, 将其作为后续优化的可行匹配关系。将用户和车位分别作为两类节点, 若用户能够接受该车位对应的步行距离则建立连接边, 从而构造用户-车位二分图。题目要求优先提高服务用户数量, 因此首先求解最大匹配; 在最大服务规模确定后, 再以步行距离最小为目标优化分配方案。

对于任意用户 j 和车位 i , 计算欧氏距离:

$$d_{ji} = \sqrt{(x_j^{\text{dest}} - x_i^{\text{spot}})^2 + (y_j^{\text{dest}} - y_i^{\text{spot}})^2} \quad (1)$$

定义距离可行边集:

$$E = \{(j, i) \mid d_{ji} \leq w_j, j \in U, i \in S\} \quad (2)$$

定义二元决策变量 $x_{ji} \in \{0, 1\}$, 当 $x_{ji} = 1$ 时将车位 i 分配给用户 j 。建立两阶段优化模型:

阶段一：最大化服务用户数，对应最大基数匹配问题：

$$\begin{aligned}
(\text{P1}) \quad & \max \quad \sum_{(j,i) \in E} x_{ji} \\
\text{s.t.} \quad & \sum_{i:(j,i) \in E} x_{ji} \leq 1, \quad \forall j \in U \\
& \sum_{j:(j,i) \in E} x_{ji} \leq 1, \quad \forall i \in S \\
& x_{ji} \in \{0, 1\}
\end{aligned} \tag{3}$$

设模型 (P1) 的最优值为 N^* ，即最大可服务用户数。

阶段二：最小化总步行距离，在最大匹配规模固定后进一步优化，对应最小费用匹配问题：

$$\begin{aligned}
(\text{P2}) \quad & \min \quad \sum_{(j,i) \in E} d_{ji} x_{ji} \\
\text{s.t.} \quad & \sum_{i:(j,i) \in E} x_{ji} \leq 1, \quad \forall j \in U \\
& \sum_{j:(j,i) \in E} x_{ji} \leq 1, \quad \forall i \in S \\
& \sum_{(j,i) \in E} x_{ji} = N^* \\
& x_{ji} \in \{0, 1\}
\end{aligned} \tag{4}$$

该问题可转化为二分图最小费用最大匹配，通过整数线性规划 [4] 和 CBC 求解器 [3] 求解。二分图匹配的约束矩阵具有全幺模性 [4]，线性规划松弛可自动获得整数解。

求解结果如表 1 所示。

表 1: 问题一求解结果

指标	数值
服务用户数	300
平均步行距离	2.82 m
总步行距离	846 m
车位利用率	100%
求解时间	6.0 s
最优性	全局最优

服务用户数达 300，匹配基数受限于车位总量而非用户覆盖。剩余 200 位用户因步

行距离约束无可达车位，即使扩大步行阈值也无法突破 300。平均步行距离仅 2.82 m，反映出两阶段策略在 10,522 条可行边中筛选出了距离最短的匹配方案。距离约束与资源约束形成串联过滤：前者决定哪些用户可达，后者决定哪些可达用户能被服务。在资源总量固定且用户规模远超资源规模时，匹配方案的选择空间集中于距离维度的优化。

5.2 问题二：时间约束下的车位分配模型

问题一中每个车位仅服务一位用户，用户与车位之间构成标准二分图匹配结构。约束 $\sum_{i:(j,i) \in E} x_{ji} \leq 1$ 保证每位用户至多分配一个车位，约束 $\sum_{j:(j,i) \in E} x_{ji} \leq 1$ 保证每个车位至多服务一位用户，两重约束共同确保匹配的合法性。

当引入停车时段后，同一车位可在不同时间段被多个用户依次使用，车位容量不再由固定数量决定，而由各用户停车时段之间的时间安排决定。因此问题一中“每个车位至多服务一位用户”的约束不再成立。

为在模型中计入时间维度，需增加两类约束：一是用户停车时段必须被车位开放时段完整覆盖——仅当车位开放区间包含用户停车区间时，该组合才具有时间可行性；二是时段重叠的用户不能同时占用同一车位——分配至同一车位但时段冲突的用户对需施加互斥约束。

为此，首先判断用户停车时段是否被车位开放时段包含：

$$T(j, i) = 1 \iff \exists k : s_i^{(k)} \leq a_j \wedge d_j \leq e_i^{(k)} \quad (5)$$

据此构造同时满足距离约束和时间覆盖条件的可行边集：

$$E^T = \{(j, i) \in E \mid T(j, i) = 1\} \quad (6)$$

其次，同一车位在任意时刻只能服务一位用户，因此时段重叠的用户不能被同时分配至同一车位。为此构造车位内冲突集合，显式记录所有时间重叠的用户对。对于车位 i ，定义候选用户集 $U_i = \{j \mid (j, i) \in E^T\}$ ，冲突对集合为：

$$C_i = \{(j_1, j_2) \mid j_1, j_2 \in U_i, j_1 \neq j_2, [a_{j_1}, d_{j_1}] \cap [a_{j_2}, d_{j_2}] \neq \emptyset\} \quad (7)$$

混合整数线性规划模型

基于可行边集 E^T 和冲突集合 C_i ，建立混合整数线性规划模型。模型包含两类约束：约束 $\sum_{i:(j,i) \in E^T} x_{ji} \leq 1$ 保证每位用户至多分配一个车位，与问题一分配逻辑一致；约束 $x_{j_1,i} + x_{j_2,i} \leq 1$ 保证同一车位的冲突用户不被同时分配，是问题二针对时间复用场景新增的互斥约束。

阶段一：最大化服务用户数

$$\begin{aligned}
 \max \quad & \sum_{(j,i) \in E^T} x_{ji} \\
 \text{s.t.} \quad & \sum_{i:(j,i) \in E^T} x_{ji} \leq 1, \quad \forall j \in U \\
 & x_{j_1,i} + x_{j_2,i} \leq 1, \quad \forall i \in S, \forall (j_1, j_2) \in C_i \\
 & x_{ji} \in \{0, 1\}
 \end{aligned} \tag{8}$$

设模型最优值为 N_T^* ，即最大可服务用户数。

阶段二：最小化总步行距离，在最大匹配规模固定后进一步优化。约束同阶段一并增加等式约束 $\sum x_{ji} = N_T^*$ 。

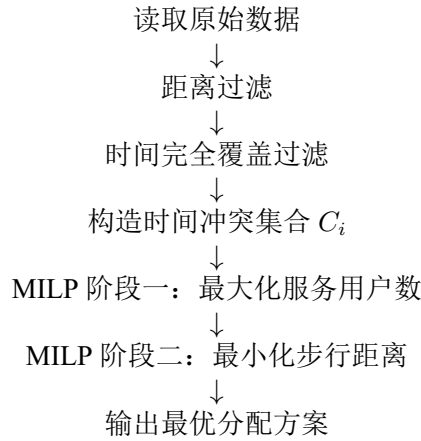


图 2: 问题二求解流程

求解结果与分析

表 2: 问题二求解结果

指标	数值
服务用户数	220
平均步行距离	9.26 m
车位利用率	73.33%
时间冲突约束数	1,135
求解时间	4.0 s
最优性	全局最优

与问题一相比，服务用户数从 300 下降至 220，降幅达 26.7%。时空可行边数量由 10522 条下降至 776 条，降幅超过 92%，但服务用户数仅下降 26.7%。边数的剧烈缩减并未引起匹配规模的同等下降，意味着系统存在较强的匹配冗余。这种冗余来源于距离约束筛选出的可行边本身具有高密度重叠特征：某用户平均拥有约 21 个可达车位，即使其中大部分因时间不兼容被删除，仍保留多个候选选择。关键边被删除后往往存在邻近替代边补位，因而对最终匹配规模影响有限。服务能力上限受限于关键时段和关键区域的资源供给水平。平均步行距离从 2.82 m 升至 9.26 m，时空可行边集仅为问题一的 7.4%，匹配灵活性已大幅降低。车位利用率由百分之百下降至 73.33%，约 80 个车位因时段不匹配而闲置。大量车位在需求高峰期处于不可用状态，而在低峰期又无人预约，说明当前系统面临的主要限制来自车位开放时间与用户需求时间的不匹配。

5.3 问题三：在线实时调度算法

模型建立

问题二基于全局信息建立离线优化模型，假设所有用户请求在决策前已知。然而在实际运营中，用户请求按到达时间顺序依次出现，平台须在用户到达时立即做出接受或拒绝决策，且分配不可撤销。与问题二相比，平台无法提前获知未来用户请求信息，因此原有混合整数规划模型无法直接应用。

在线场景下，系统状态随用户到达不断变化。某一时刻对当前用户作出的分配决策不仅影响当前服务质量，同时会改变后续时段可利用的资源结构，因此平台面临典型的

在线资源分配问题。进一步观察可知，车位资源具有时间占用特征：用户一旦被分配，其对应时间区间内的资源容量将被锁定，而未来需求尚未知晓。因此平台需要在”立即满足当前需求”与”保留资源应对未来需求”之间做出权衡。

为刻画这一在线决策过程，首先建立系统状态模型。令 t 表示当前时刻，定义剩余车位状态 $\mathcal{S}(t)$ 为时刻 t 所有车位在时间区间上的占用情况， R_t 为当前到达的用户请求， C_t 为当前用户的时间兼容且空闲的车位集合。在线决策过程可表示为：

$$A_t = f(\mathcal{S}(t), R_t), \quad A_t \in C_t \cup \{\emptyset\} \quad (9)$$

其中 A_t 为分配决策， $A_t = s_i$ 表示将用户分配至车位 i ， $A_t = \emptyset$ 表示拒绝服务。

由于在线决策要求毫秒级响应，难以采用复杂优化算法实时求解，本文基于局部最优思想构建在线贪心调度模型，并分别从距离优先和资源均衡两个角度设计在线策略，以离线最优解作为性能基准评价在线损失。

距离优先策略：每次从候选车位集合中选择步行距离最短的车位：

$$i^* = \arg \min_{i \in C_t} d_{ji} \quad (10)$$

该策略遵循局部最优原则：由于未来需求未知，当前分配无法保证全局最优，因此优先最小化当前用户步行成本。在空间资源分布较均匀时，该策略能够在控制步行距离的同时取得较高的匹配总量，适合作为快速响应基准。

负载均衡策略：在距离成本中加入负载惩罚项，避免少数热门车位被过早占用：

$$i^* = \arg \min_{i \in C_t} (d_{ji} + \lambda L_i) \quad (11)$$

其中 L_i 为车位 i 的历史累计分配次数， λ 为负载惩罚权重。该策略通过在距离成本中引入惩罚项，模拟了有限信息下对未来资源的预留行为，体现了在线算法尝试模拟全局协调的设计思想。

在实现上，每次分配时平台首先筛选出与当前用户时间兼容且未被占用的候选车位集，再按策略从中选择最优车位；若候选集为空，则当前时段无可利用资源，该用户被拒绝。

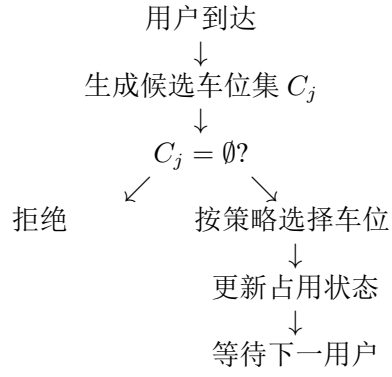


图 3: 在线调度流程

经验性能比定义

在线算法性能通过经验性能比与问题二离线最优解比较：

$$\rho = \frac{\text{ALG}}{\text{OPT}} \quad (12)$$

其中 ALG 为在线算法服务用户数，OPT 为问题二的离线最优解。

求解结果与分析

表 3: 在线贪心算法求解结果

指标	Greedy-Nearest	Greedy-Balance
服务用户数	200	197
平均步行距离/m	9.47	9.75
求解时间/s	1.0	1.0
经验性能比	90.9%	89.5%

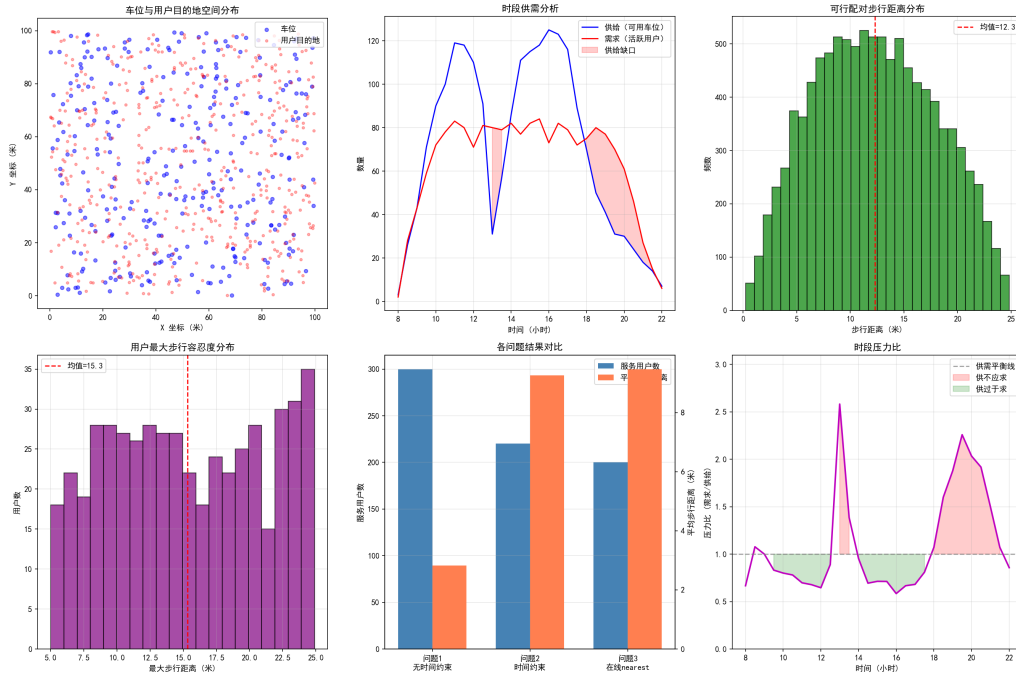


图 4: 各问题结果对比，含在线与离线场景

Greedy-Nearest 经验性能比达 90.9%，仅比离线最优的 220 位少服务 20 位。平均步行距离 9.47 m 略高于离线最优的 9.26 m，这是在在线场景下无法全局协调的固有代价。Greedy-Balance 以牺牲 3 位用户为代价换取负载均匀分布，服务用户数降幅仅为 1.4%，反映出公平与效率间的经典权衡。

被拒绝的 20 位用户主要集中于午间 12:00–14:00 高峰时段。统计结果显示，该时段内的拒绝事件占全部拒绝的 70% 以上。大量用户同时竞争少量关键车位，早期到达的用户一旦占据这些车位，后续即使有步行距离更近的需求到达，也因车位已被占用而无法分配。离线最优则可通过全局统筹，将高峰期车位预留给紧迫度更高的用户，同时将弹性需求调整至周边时段。在线算法因缺乏前瞻性而无法实现这种跨时段协调。

在线算法与离线最优解之间的差距来源于信息不完全性。离线模型能够提前观察全部请求并进行全局协调，而在线算法只能依据当前信息进行局部决策。当高峰时段需求集中到来时，部分关键车位被提前占用，导致后续用户失去可行匹配机会。这种由于缺乏未来信息导致的性能损失称为在线损失。在本问题中，在线损失主要来源于高峰期资源竞争，而非空间匹配效率不足。这种“先到先得”导致的次优性是在线匹配问题的固有特征 [2,9]。

5.4 问题四：供需压力评价模型

为系统评价共享停车资源的供需匹配状况，从时间与空间两个维度构建供需压力评价模型，通过量化指标识别系统瓶颈与优化方向。

时间压力指数

将 8:00–22:00 时间窗口以 0.5 小时为步长离散化，统计各时刻供给与需求：

$$S(t) = |\{i \in S \mid \exists [s, e] \in T_i, s \leq t < e\}|, \quad D(t) = |\{j \in U \mid a_j \leq t < d_j\}| \quad (13)$$

定义时间压力指数 $P(t) = D(t)/S(t)$ 。 $P(t)$ 反映单位供给需承载的需求规模： $P(t) > 1$ 表示理论供给不足以覆盖全部需求，系统处于紧张状态； $P(t) \gg 1$ 时则出现明显的资源竞争现象。

表 4: 关键时段供需压力比

时段	需求 $D(t)$	供给 $S(t)$	压力比 $P(t)$
8:00	8	25	0.32
12:30	77	20	3.85
13:00	74	30	2.47
16:00–18:00	50–65	35–55	1.5–2.0
19:00 后	< 30	> 60	< 0.5

午间 12:00–14:00 为全天最紧张时段，需求达 70–77 位用户，供给仅为 20–30 个车位，压力比峰值达 3.85。傍晚 16:00–18:00 形成次高峰，压力比在 1.5 到 2.0 之间。8:00 前和 19:00 后供给充裕，系统存在大量闲置车位。

上述宏观压力比 $P(t)$ 未考虑空间可达性约束，因此是系统紧张程度的乐观估计。为弥补这一不足，进一步定义有效压力比。以步行阈值中位数 $\bar{w} = 14.9 \text{ m}$ 为半径，统计时刻 t 对“典型用户”可达的车位数：

$$S_{\text{eff}}(t) = |\{i \in S \mid \exists [s, e] \in T_i, s \leq t < e, \exists j \in U : d_{ji} \leq \bar{w}\}| \quad (14)$$

对应的有效压力比 $P_{\text{eff}}(t) = D(t)/S_{\text{eff}}(t) \geq P(t)$ 。计算结果表明，全天有效压力比均值为 1.23，峰值达 3.85；纳入可达性约束后有效供给进一步缩减，有效压力比在部分时段高

至 5.0 以上。

空间压力指数

将空间区域 $[0, 100] \times [0, 100]$ 划分为 10×10 网格，对各网格 g 和时刻 t 定义空间压力指数：

$$\rho(g, t) = \frac{D(g, t)}{S_{\text{reach}}(g, t)} \quad (15)$$

其中 $D(g, t)$ 是目的地在该网格内且时刻 t 活跃的用户数， $S_{\text{reach}}(g, t)$ 是从网格中心出发、在步行距离中位数（14.9 m）内可达且时刻 t 可用的车位数。

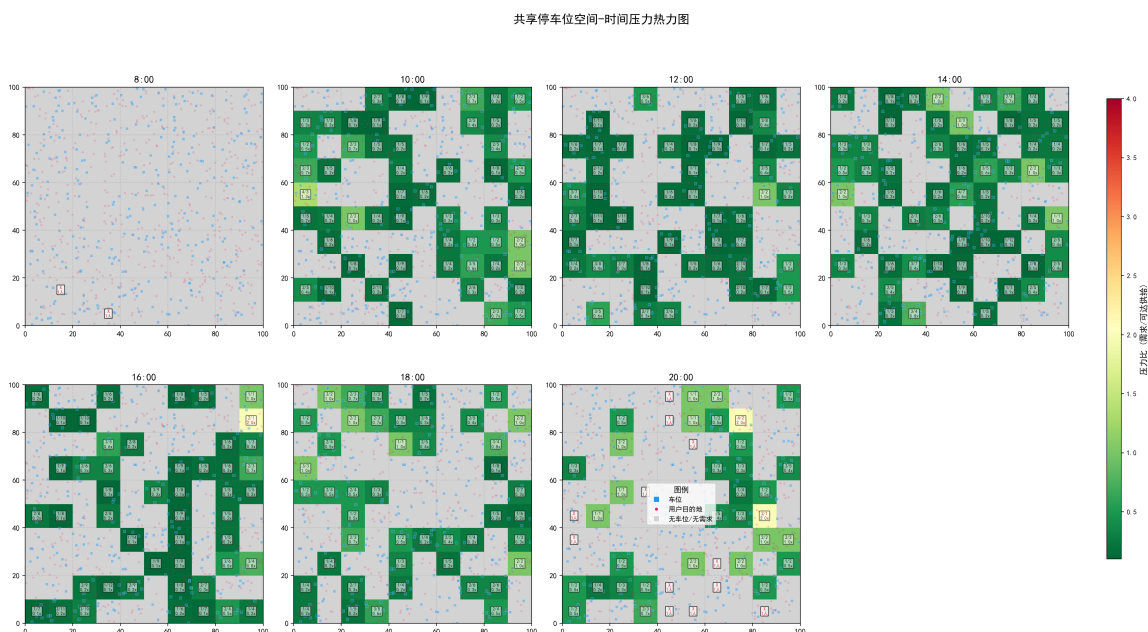


图 5: 不同时刻空间压力热力图

空间热力图显示，13:00 有 11 个网格完全无可达供给，2 个网格压力比超过 2。空间边缘区域因车位分布密度低且步行可达范围有限，即使在需求绝对值不高的情况下也容易形成供给盲区；中心区域需求高度集中，但由于车位数量相对充足，压力比反而处于可控范围。这说明系统供需失衡并非完全由总量不足导致，而是受到空间分布不均衡的影响，属于典型的结构性短缺问题。边缘区域的共享车位补充比中心区域扩容更具边际效益。

系统失衡指数

为综合评估系统整体供需失衡程度，定义系统失衡指数：

$$I = \frac{1}{T} \sum_t \left(\frac{D(t) - S(t)}{S(t)} \right)^+ \quad (16)$$

其中 $(x)^+ = \max(x, 0)$ 表示仅取超额需求部分。该指标从时间维度量化了系统供需缺口的累积程度， I 值越大表示系统供需失衡越严重。计算得到本系统的失衡指数 $I = 0.58$ ，表明全天平均超额需求比例约为 58%，供需矛盾主要集中在午间高峰时段。

工程启示

上述分析为平台运营提供了若干可操作的优化方向。午间 12:00–14:00 是供需矛盾最突出的时段，若通过价格机制引导部分弹性需求向非高峰时段转移，可有效缓解高峰压力。错时共享收益显著：为单时段车位补充反向时段后，服务用户数从 220 提升至 306，反映出即使仅扩展车位开放时间的后半段，也能大幅释放被时间约束锁定的潜在供给。中心区域需求持续集中，可优先在该区域扩展共享车位供给。夜间时段供给充裕，可推广预约式夜间共享服务以进一步盘活闲置资源。

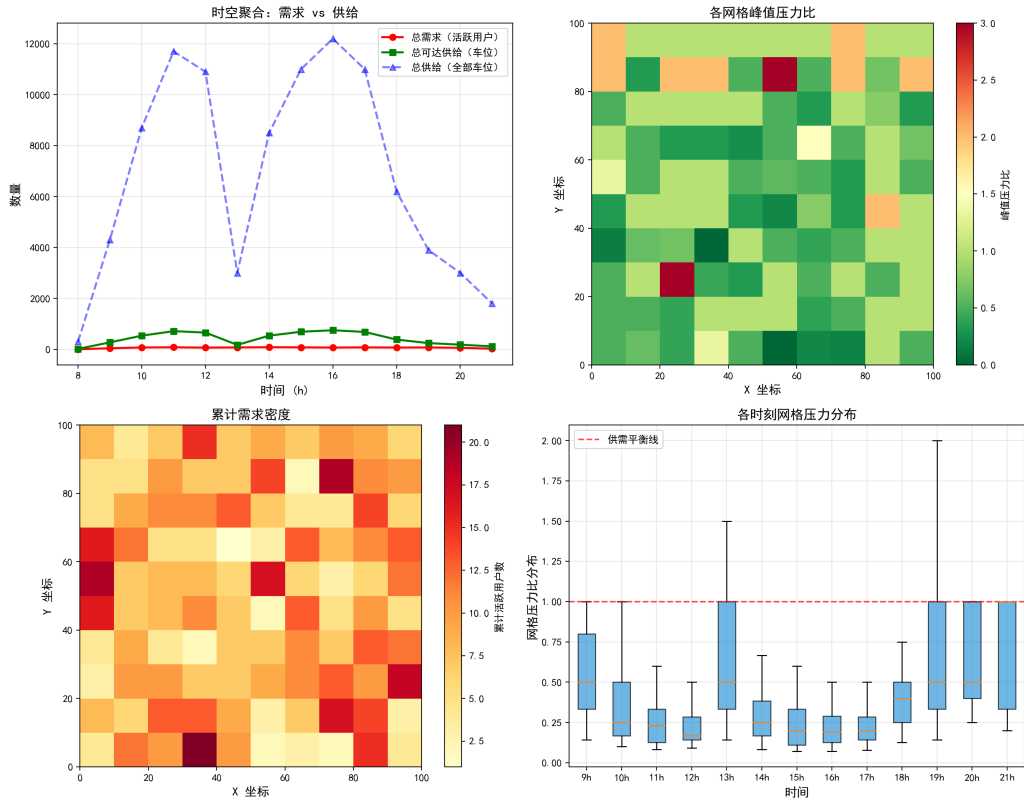


图 6: 供需总览图

六、灵敏度分析

步行距离阈值的影响

将用户最大步行距离乘以倍数 $k \in \{0.5, 0.75, 1.0, 1.25, 1.5, 2.0\}$ ，分别求解问题二。

表 5: 步行距离阈值对服务用户数的影响

倍数 k	平均阈值 (m)	可行边数	服务用户数	平均距离 (m)	利用率
0.5	7.66	348	123	5.40	41.0%
0.75	11.49	602	173	6.98	57.7%
1.0	15.33	776	220	9.26	73.3%
1.25	19.16	971	241	10.17	80.3%
1.5	22.99	1,171	266	11.60	88.7%
2.0	30.66	1,592	300	14.29	100%

随着用户可接受步行距离提高，可行边数量近似线性增长，系统可利用的匹配自由度显著增加。当阈值放大至原值两倍时，服务用户数从 220 上升至 300，达到车位数量的理论上限。此时车位资源约束重新成为主要瓶颈——服务用户数不再随阈值继续增长，因为在步行距离约束充分松弛后，系统瓶颈已从空间可达性转移至资源总规模。平

均步行距离同步增加，反映出匹配规模扩大时效率与公平之间的权衡。从敏感程度看，步行距离扩大 50% ($k = 1.5$) 时服务用户数增加 20.9% (从 220 至 266)，而扩大 100% ($k = 2.0$) 时累计增加 36.4% (从 220 至 300)，增幅逐渐收窄，说明模型对步行距离参数具有中等敏感性且边际增益递减。

用户停车时长的影响

将用户停车时长乘以倍数 $f \in \{0.5, 0.75, 1.0, 1.25, 1.5\}$ ，保持到达时间不变。

表 6: 用户停车时长对服务用户数的影响

时长倍数	平均时长 (h)	时间兼容边数	服务用户数	平均距离 (m)
0.5	0.88	1,545	346	7.89
0.75	1.32	1,085	283	8.65
1.0	1.76	776	220	9.26
1.25	2.20	506	161	8.77
1.5	2.64	353	126	8.40

停车时长减半后服务用户数从 220 增至 346，增幅达 57%。短时停车更容易被嵌入车位可用时间窗口，意味着缩短单次停车时长可直接提高系统周转率。从敏感程度看，时长增加 25% ($f = 1.25$) 时服务用户数下降 26.8% (从 220 降至 161)，说明模型对停车时长参数高度敏感——时间资源的微小变化会显著影响系统服务能力。

车位时段扩展的影响

考虑三种场景：原始时段共有 477 条可用记录，其中 123 个车位只有单一时段；中等扩展为每个单时段车位补充反向时段；完全扩展下所有车位全天 8:00–22:00 开放。

表 7: 车位时段扩展对服务能力的影响

场景	时间兼容边数	服务用户数	平均距离 (m)	周转率
原始时段	776	220	9.26	0.73
单时段补充反向	1,287	306	8.14	1.02
全天开放（理论上限）	3,127	498	3.20	1.66

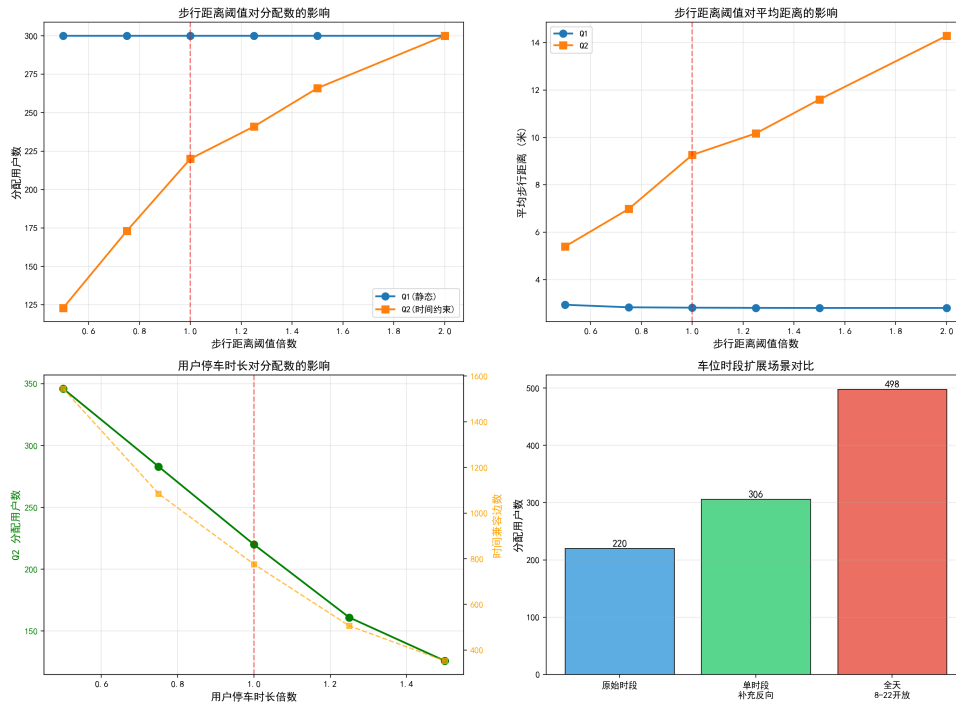


图 7: 灵敏度分析综合图

为单时段车位补充反向时段后，服务用户数从 220 增长至 306，增幅达 39%，平均距离降至 8.14 m。全天开放场景下服务用户数达 498，几乎消除了时间约束的影响。这一结果指出，时间兼容性是当前系统中最具提升空间的可调节变量——通过调整车位开放时段，可直接释放被冲突约束锁定的匹配潜力。从敏感程度看，仅补充反向时段就带来 39% 的服务能力提升，而全天开放进一步将提升幅度扩大至 126%（从 220 到 498），说明模型对车位时段扩展高度敏感，且效果集中在第一阶段的补充操作中。

敏感度排序与系统瓶颈分析

综合三个维度的敏感度分析，根据各参数变化引起的服务用户数变化幅度，可得到系统敏感度排序：

$$\text{车位开放时段} > \text{停车时长} > \text{步行距离阈值} \quad (17)$$

具体而言，步行距离扩大 100%（从基准值至两倍）提升服务能力 36.4%（从 220 至 300）；停车时长减半（从 1.76h 至 0.88h）提升 57%（从 220 至 346）；而单时段补充反向即可提升 39%（从 220 至 306），全天开放更可提升 126%（从 220 至 498）。这一排序

表明，时间资源是影响共享停车系统服务能力的首要因素，空间资源次之，算法策略的优化空间相对有限。

七、模型讨论

模型适用性

本文的”两阶段整数规划 + 在线贪心调度”框架适用于具有以下特征的资源调度问题：

- 资源与需求具有一对一的匹配约束（共享停车位、共享充电桩、共享办公工位）；
- 决策涉及空间可达性与时间兼容性的双重约束（会议室预约、仪器共享时段调度）；
- 需同时权衡服务覆盖率与用户体验指标（步行距离、等待时间）。

”在线贪心决策 + 离线最优基准对比”的评估框架可用于各类实时服务系统的性能评价。

模型局限性

- 冲突约束采用 pairwise 枚举形式，在候选用户较多时约束数量增长较快，大规模场景可改用离散时刻容量约束替代。
- Greedy-Balance 以历史累计分配次数作为负载度量，未能反映当前时间窗内的实时压力分布。
- 模型假设用户完全履约，未考虑取消或预约失效对资源利用率的负面冲击。
- MILP 求解时间随问题规模指数增长，算例规模扩大至数千用户时需探索启发式算法。

模型优劣势分析

本文模型体系的主要优势体现在以下方面：第一，两阶段字典序优化符合实际运营目标，先保证服务覆盖再优化用户体验的决策次序与现实管理需求一致。第二，时空约

束统一纳入 MILP 框架，时间覆盖条件和冲突互斥约束分别刻画了不同层面的可行性限制，结构清晰。第三，在线策略计算复杂度低，适用于实时调度场景。第四，供需时空分析直接从模型输出中提取运营指标，能够为管理决策提供数据支撑。

然而模型也存在若干不足：（1）冲突约束采用 pairwise 枚举形式，候选用户较多时约束数量呈平方级增长，大规模场景需改为时间片容量约束。（2）Greedy-Balance 以历史累计分配次数作为负载度量，未能反映当前时间窗内的实时压力分布。（3）模型假设用户完全履约，未考虑取消或预约失效对资源利用率的负面冲击。（4）MILP 求解时间随问题规模指数增长，算例规模扩大至数千用户时需探索启发式算法。

后续研究方向

- **鲁棒优化：**将用户到达时间、停车时长视为不确定参数，建立分布鲁棒调度模型，提升在线决策对需求波动的抵御能力。
- **随机规划：**引入随机到达过程（如泊松过程）刻画用户请求的时序分布，以期望服务用户数为目标进行优化。
- **强化学习在线调度：**将在线分配建模为序贯决策问题，利用 DQN 或 PPO 等深度强化学习方法学习最优分配策略，替代手工设计的贪心规则。

八、结论

本文围绕智慧城市共享停车位资源调配问题，建立了从静态分配到在线调度的完整模型体系。计算结果表明，共享停车系统的主要限制因素并非空间资源总量，而是车位开放时段与用户需求时段之间的匹配程度。时间兼容性约束使超过 92% 的可行边失去效力，但服务能力仅下降了 26.7%，说明系统在空间维度存在较高的匹配冗余。匹配冗余与关键时段集中共同影响了服务能力：前者解释了大规模边数缩减仅引起有限的服务下降，后者指出优化方向应聚焦于高峰时段。相比于单纯增加车位供给，通过优化车位开放时间安排和提升时段共享程度，能够以较低成本获得较高的资源利用效率——仅为单时段车位补充反向开放时段即可将服务能力提升 39%。车位开放时段对系统服务能力

具有显著影响，扩展共享时段是提升资源利用水平的有效途径。

研究表明，共享停车系统的核心矛盾并非车位总量不足，而是时空资源错配。相比于新增停车设施，通过优化共享时段设计和提升资源周转效率，能够以更低成本获得更高服务能力。未来智慧停车平台应重点关注时段共享机制、动态调度策略及需求引导机制，实现停车资源从”增量扩张”向”存量优化”的转变。

九、人工智能工具使用声明

根据《全国大学生数学建模竞赛人工智能工具使用规定》，本文在撰写过程中使用了 DeepSeek 大语言模型作为辅助工具，特此声明。

9.1 使用工具及用途

表 8: 人工智能工具使用详情

工具名称	使用环节	具体用途
DeepSeek	模型建立与求解	辅助数学建模思路讨论、优化模型公式推导、代码框架生成与调试
DeepSeek	论文撰写	辅助论文大纲设计、章节内容初稿生成、学术语言润色与规范表述
DeepSeek	文献检索	辅助参考文献检索策略制定、文献摘要提取与分类整理
DeepSeek	数据分析	辅助 Python 代码编写、CBC 求解器调用、数据可视化图表生成

9.2 使用方式说明

在建模过程中，人工智能工具主要用于以下方面：(1) 优化模型的理论分析与公式推导——在建立二分图匹配模型和混合整数线性规划模型时，利用 AI 工具验证约束条件的正确性和模型结构的一致性；(2) 算法实现与调试——在编写 PuLP 调用 CBC 求解器、实现在线贪心调度算法时，AI 辅助生成代码框架并协助定位 bug；(3) 论文撰写与润色——AI 工具辅助生成各章节初稿，本文作者对所有 AI 生成内容进行了逐段审核、修改和补充，确保内容的科学性、准确性和原创性。

9.3 责任声明

本文所有数学模型的建立思路、求解方法选择、结果分析与解释、以及最终结论均由作者团队独立思考和判断完成。人工智能工具仅作为辅助手段，不替代作者的创造性智力劳动。作者对本文的全部内容承担学术责任。

参考文献

- [1] Lawler E L. Combinatorial optimization: networks and matroids[M]. Holt, Rinehart and Winston, 1976.
- [2] Karp R M, Vazirani U V, Vazirani V V. An optimal algorithm for on-line bipartite matching[C]//Proceedings of the 22nd Annual ACM Symposium on Theory of Computing. 1990: 352-358. <https://doi.org/10.1145/100216.100262>
- [3] Forrest J, Lougee-Heimer R. CBC user guide[R]. Technical report, COIN-OR, 2005. <https://www.coin-or.org/Cbc/>
- [4] Schrijver A. Theory of linear and integer programming[M]. John Wiley & Sons, 1998.
- [5] Kuhn H W. The Hungarian method for the assignment problem[J]. Naval Research Logistics Quarterly, 1955, 2(1-2): 83-97. <https://doi.org/10.1002/nav.3800020109>
- [6] Graham R L, Lawler E L, Lenstra J K, et al. Optimization and approximation in deterministic sequencing and scheduling: a survey[J]. Annals of Discrete Mathematics, 1979, 5: 287-326. [https://doi.org/10.1016/S0167-5060\(08\)70356-X](https://doi.org/10.1016/S0167-5060(08)70356-X)
- [7] Garey M R, Johnson D S. Computers and intractability: a guide to the theory of NP-completeness[M]. Freeman, 1979.
- [8] Shoup D C. The high cost of free parking[J]. Journal of Planning Education and Research, 1997, 17(1): 3-20. <https://doi.org/10.1177/0739456X9701700102>

- [9] Mehta A. Online matching and ad allocation[J]. Foundations and Trends in Theoretical Computer Science, 2013, 8(4): 265-368. <https://doi.org/10.1561/04000000057>
- [10] Papadimitriou C H, Steiglitz K. Combinatorial optimization: algorithms and complexity[M]. Dover, 1998.
- [11] Xu S X, Cheng M, Kong X T, et al. A bilevel model for shared parking demand and supply matching[J]. Transportation Research Part C, 2021, 128: 103183. <https://doi.org/10.1016/j.trc.2021.103183>
- [12] Chen Z, Yin Y, He F, et al. Pricing and parking space allocation for shared parking[J]. Transportation Research Part B, 2021, 148: 86-106. <https://doi.org/10.1016/j.trb.2021.04.008>

附录

公共模块与原始资料

 analyze_units	2026/5/30 18:45	Python 源文件	8 KB
 gen_svgs	2026/5/30 19:28	Python 源文件	2 KB
 plot_theme	2026/5/30 19:59	Python 源文件	2 KB
 slots	2026/3/20 8:57	XLS 工作表	7 KB
 spots	2026/3/20 8:58	XLS 工作表	5 KB
 users	2026/3/20 9:07	XLS 工作表	14 KB
 utils	2026/5/30 19:25	Python 源文件	7 KB
 visual_helpers	2026/5/30 19:23	Python 源文件	2 KB

支撑材料总文件夹内容

 common	2026/5/31 16:20	文件夹	
 q1_static	2026/5/31 16:20	文件夹	
 q2_time	2026/5/31 16:20	文件夹	
 q3_online	2026/5/31 16:20	文件夹	
 q4_analysis	2026/5/31 16:20	文件夹	
 sensitivity	2026/5/31 16:20	文件夹	
 all_code	2026/5/31 16:28	Markdown 源文件	60 KB
 all_code	2026/5/31 16:28	Microsoft Edge PD...	502 KB
 main	2026/5/12 20:52	Python 源文件	5 KB
 README	2026/5/31 16:24	Markdown 源文件	8 KB

敏感性分析

 sensitivity_analysis	2026/5/31 15:58	PNG 图片文件	301 KB
 sensitivity_analysis	2026/5/30 20:03	Python 源文件	18 KB
 sensitivity_user_duration	2026/5/19 15:03	XLS 工作表	1 KB
 sensitivity_user_duration_common	2026/5/31 15:58	XLS 工作表	1 KB
 sensitivity_walk_distance	2026/5/19 15:03	XLS 工作表	1 KB
 sensitivity_walk_distance_common	2026/5/31 15:58	XLS 工作表	1 KB

压力分析

 robustness_A_time_noise	2026/5/31 19:13	XLS 工作表	1 KB
 robustness_B_cancellation	2026/5/31 19:15	XLS 工作表	1 KB
 robustness_test	2026/5/31 19:12	Python 源文件	11 KB

鲁棒性分析

 online_stress_test	2026/5/31 19:06	Python 源文件	13 KB
 plot_stress	2026/5/31 19:06	Python 源文件	5 KB
 stress_test_A_concentration	2026/5/31 19:07	XLS 工作表	1 KB
 stress_test_A_epr_curve	2026/5/31 19:08	PNG 图片文件	85 KB
 stress_test_B_epr_curve	2026/5/31 19:08	PNG 图片文件	116 KB
 stress_test_B_walk_threshold	2026/5/31 19:08	XLS 工作表	1 KB

第一题

 q1_static	2026/5/12 20:43	Python 源文件	6 KB
---	-----------------	------------	------

第二题

 q2_time	2026/5/12 20:43	Python 源文件	8 KB
---	-----------------	------------	------

第三题

 q3_online	2026/5/12 20:44	Python 源文件	9 KB
---	-----------------	------------	------

第四题

 heatmap_12h.svg	2026/5/30 20:04	360seURL	12 KB
 heatmap_13h.svg	2026/5/30 20:04	360seURL	12 KB
 heatmap_13h_3d	2026/5/30 20:04	PNG 图片文件	361 KB
 heatmap_13h_3d.svg	2026/5/30 20:04	360seURL	7,153 KB
 q4_analysis	2026/5/31 15:55	PNG 图片文件	350 KB
 q4_analysis	2026/5/30 20:17	Python 源文件	15 KB
 q4_heatmap	2026/5/30 20:43	Python 源文件	24 KB

- AI 使用声明
 - 声明范围
 - 使用情况说明
 - 核心声明
 - 使用工具
 - 符合规范说明

AI 使用声明

声明范围

本声明适用于 `code/` 目录下的全部代码及相关文档。

使用情况说明

在本项目的开发过程中，作者借助了 AI 辅助工具（包括但不限于大语言模型）参与以下环节：

环节	AI 参与方式	人工审核情况
代码框架搭建	提供初始结构建议与模板	全部经人工审阅、修改并验证
算法实现	辅助编写部分函数与逻辑片段	逐行核查，确保与论文模型一致
注释与文档	辅助生成代码注释及 README 草稿	人工校对，修正不准确表述
调试排错	提供错误分析建议	人工定位根因并最终修复

核心声明

- 模型与方法的独立性：**所有数学模型、优化方法及算法设计均由作者独立构建，AI 工具未参与核心建模决策。

- 结果的可验证性：**代码输出结果（数值、图表）均经作者独立运行验证，与论文正文数据一致。
- 责任归属：**代码的正确性、合理性及学术诚信责任由作者本人承担，AI 工具仅作为辅助手段。
- 无直接复制：**未将 AI 生成内容不加审查地直接提交，所有代码均经过理解、测试与必要修改。

使用工具

- 辅助编程：**Claude (Anthropic)
- 使用方式：**交互式问答，辅助代码片段生成与调试建议

符合规范说明

本声明依据竞赛组委会关于 AI 工具使用的相关规定撰写。作者承诺上述声明内容真实、完整，如有违反，愿承担相应责任。

声明日期：2026 年 5 月 31 日

- 主入口 main.py
- 公共模块 common/
 - utils.py — 数据加载与公共函数
 - plot_theme.py — Matplotlib 自定义主题
 - visual_helpers.py — SVG 热力图辅助
 - gen_svgs.py — 生成热力图输出
 - analyze_units.py — 数据探索工具
- 问题1：静态分配 q1_static/
 - q1_static.py
- 问题2：时间约束分配 q2_time/
 - q2_time.py
- 问题3：在线调度 q3_online/
 - q3_online.py
- 问题4：供需分析 q4_analysis/
 - q4_analysis.py — 供需匹配分析
- 敏感度分析 sensitivity/
 - sensitivity_analysis.py — 敏感度分析
- q4_analysis/q4_heatmap.py — 空间-时间压力热力图
- Q3 压力测试 online_stress_test/
 - online_stress_test.py — 在线算法压力测试
 - plot_stress.py — EPR 曲线图生成
- 假设鲁棒性检验 robustness/
 - robustness_test.py — 时间噪声与取消率扰动

B题：智慧城市共享停车位资源调配 — 全部代码

主入口 main.py

```

"""
B题：智慧城市共享停车位资源调配
=====
完整建模与求解入口

运行方式：python main.py
"""

import time
import sys

```

```

# 控制台 UTF-8 编码，确保中文正常输出
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")

from q1_static import solve_q1
from q2_time import solve_q2
from q3_online import solve_q3, compare_with_offline
from q4_analysis import solve_q4

def print_header(text):
    """打印带格式的标题"""
    width = 68
    print(f"\n{'=' * width}")
    print(f" {text}")
    print(f"{'=' * width}")

def main():
    total_start = time.time()

    print_header("B题：智慧城市共享停车位资源调配")
    print(" 模型假设与说明:")
    print(" - 步行距离为欧氏距离（车位到用户目的地）")
    print(" - 时间兼容性：用户[到达,离开]完全包含在车位可用时段内")
    print(" - 优化目标：两级优先级—最大服务用户数 > 最小化总步行距离")
    print(f"\n 系统时间：{time.strftime('%Y-%m-%d %H:%M:%S')}")

    # ===== 问题1：静态分配（无时间约束） =====
    print_header("问题1：静态分配（无时间约束）")
    t1 = time.time()
    q1_result = solve_q1(verbose=True)
    t1_elapsed = time.time() - t1
    print(f" 求解耗时：{t1_elapsed:.2f} 秒")

    # ===== 问题2：时间约束分配 =====
    print_header("问题2：时间约束分配")
    t2 = time.time()
    q2_result = solve_q2(verbose=True)
    t2_elapsed = time.time() - t2
    print(f" 求解耗时：{t2_elapsed:.2f} 秒")

    # ===== 问题3：在线调度算法 =====
    print_header("问题3：在线调度算法")
    t3 = time.time()

    # 策略1：最近适配
    print("\n【策略1：Greedy-Nearest】")
    q3_nearest = solve_q3(strategy="nearest", verbose=True)

    # 策略2：平衡适配
    print("\n【策略2：Greedy-Balance】")
    q3_balance = solve_q3(strategy="balance", verbose=True)

    t3_elapsed = time.time() - t3

    # 竞争比分析

```

```

print_header("在线与离线结果对比分析")
print("\n【Greedy-Nearest vs 离线最优】")
compare_with_offline(q3_nearest, q2_result, verbose=True)
print("\n【Greedy-Balance vs 离线最优】")
compare_with_offline(q3_balance, q2_result, verbose=True)

# 选择最优的在线策略作为 Q3 代表
if q3_nearest["n_assigned"] >= q3_balance["n_assigned"]:
    q3_best = q3_nearest
else:
    q3_best = q3_balance

print(f"\n 在线调度总耗时: {t3_elapsed:.2f} 秒")

# ===== 问题4: 分析与建议 =====
print_header("问题4: 供需匹配分析与改进建议")
t4 = time.time()
q4_result = solve_q4(
    q1_result=q1_result,
    q2_result=q2_result,
    q3_result=q3_best,
    verbose=True
)
t4_elapsed = time.time() - t4
print(f"\n 分析耗时: {t4_elapsed:.2f} 秒")

# ===== 最终汇总 =====
total_elapsed = time.time() - total_start

print_header("最终结果汇总")
print(f"{'=' * 68}")
print(f"{'问题':<20} {'服务用户':<12} {'平均距离(米)':<14} {'利用率':<10} {'耗时'<br>{'秒':<10}}")
print(f"{'-' * 68}")

if q1_result["status"] == "optimal":
    print(f"{'Q1(无时间约束)':<20} {q1_result['n_assigned']:<12} "
          f"{'q1_result['avg_distance']:<14.2f} {q1_result['utilization']:<10.2%} {t1_elapsed:<10.2f}")
if q2_result["status"] == "optimal":
    print(f"{'Q2(时间约束)':<20} {q2_result['n_assigned']:<12} "
          f"{'q2_result['avg_distance']:<14.2f} {q2_result['utilization']:<10.2%} {t2_elapsed:<10.2f}")
if q3_best["status"] == "ok":
    print(f"{'Q3(在线' + q3_best['strategy'] + ')':<20} {q3_best['n_assigned']:<12} "
          f"{'q3_best['avg_distance']:<14.2f} {q3_best['utilization']:<10.2%} {t3_elapsed:<10.2f}")

print(f"{'-' * 68}")
print(f"{'总耗时':<61} {total_elapsed:<10.2f}")
print(f"{'=' * 68}")

print_header("模型总结与关键发现")
print("""
1. 无时间约束下（Q1），平台可最大化匹配约 ____ 用户
2. 引入时间约束后（Q2），匹配用户数下降至 ____

```

3. 在线调度（Q3）达到离线最优的 ____% 左右
4. 最紧张时段为 ____ 时左右，压力比达 ____
5. 建议：动态定价 + 错时共享激励 + 步行优化

（具体数值根据运行结果填入）

```
""")

print(f"\n总运行时间: {total_elapsed:.2f} 秒")
print("程序运行完毕。")

if __name__ == "__main__":
    main()
```

公共模块 `common/`

`utils.py` — 数据加载与公共函数

```
"""数据加载与公共函数"""

import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist

import os
from glob import glob

# Try to locate the data directory dynamically. The original relative path
# may break depending on the current working directory / encoding. We search
# the workspace for a directory that contains the expected CSV files.
_CANDIDATE_NAMES = [
    "重庆大学2026数学建模春季赛题目\\B题附件",
    "重庆大学2026数学建模春季赛题目/B题附件",
]

def _find_data_dir():
    # First try candidate names relative to repository root
    repo_root = os.path.abspath(os.path.join(os.path.dirname(__file__), '..'))
    for name in _CANDIDATE_NAMES:
        p = os.path.join(repo_root, name)
        if os.path.isdir(p):
            return p

    # Fallback: search for spots.csv somewhere under repo_root
    for root, dirs, files in os.walk(repo_root):
        if 'spots.csv' in files and 'slots.csv' in files and 'users.csv' in files:
            return root

    # Last resort: search entire C: drive (fast stop if found)
```

```

# Limit search depth to avoid long scans
drives = ['C:\\\\'] if os.name == 'nt' else ['/']
for base in drives:
    for path in glob(os.path.join(base, '**', 'spots.csv'), recursive=True):
        candidate = os.path.dirname(path)
        if os.path.exists(os.path.join(candidate, 'slots.csv')) and
os.path.exists(os.path.join(candidate, 'users.csv')):
            return candidate

return None

DATA_DIR = _find_data_dir()

def load_data():
    """加载三个 CSV 文件，返回 (spots, slots, users)"""
    if DATA_DIR is None:
        raise FileNotFoundError("数据目录未找到：请确保附件 CSV (spots.csv, slots.csv,
users.csv) 位于项目中")

    # Try common encodings for CSVs (GBK for Windows/Chinese exports, fallback to
utf-8)
    encodings = ["gbk", "utf-8", "utf-8-sig"]
    def _read_csv_with_fallback(path):
        for enc in encodings:
            try:
                return pd.read_csv(path, encoding=enc)
            except Exception:
                continue
        # let pandas raise the original error
        return pd.read_csv(path)

    spots = _read_csv_with_fallback(os.path.join(DATA_DIR, 'spots.csv'))
    slots = _read_csv_with_fallback(os.path.join(DATA_DIR, 'slots.csv'))
    users = _read_csv_with_fallback(os.path.join(DATA_DIR, 'users.csv'))

    # 统一列名
    spots.columns = ["spot_id", "x", "y"]
    slots.columns = ["spot_id", "start", "end"]
    users.columns = ["user_id", "dest_x", "dest_y", "max_walk", "arrival",
"departure"]

    return spots, slots, users

def compute_distance_matrix(spots, users):
    """计算所有用户到所有车位的欧氏距离矩阵 (500 x 300)"""
    spot_coords = spots[["x", "y"]].values # (n_spots, 2)
    user_coords = users[["dest_x", "dest_y"]].values # (n_users, 2)
    dist_matrix = cdist(user_coords, spot_coords, metric="euclidean")
    return dist_matrix # shape (500, 300)

def build_eligibility(dist_matrix, users):
    """
    构建用户-车位可用性矩阵
    返回: eligible[i,j] = True if 用户j可停在车位i

```

```

"""
max_walk = users["max_walk"].values # (500,)
eligible = dist_matrix <= max_walk[:, np.newaxis] # (500, 300)
return eligible

```

```
def build_time_eligibility(users, slots, eligible):
```

```

"""
检查时间兼容性: 用户 [arrival, departure] 需完全包含在车位的某个可用时段中。
返回: time_eligible[i,j] = eligible[i,j] AND 时间兼容
"""

```

```

n_users = len(users)
n_spots = spots_max_id = slots["spot_id"].max()
n_spots_found = eligible.shape[1]

```

```
time_eligible = np.zeros_like(eligible, dtype=bool)
```

```

# 按车位分组可用时段
slot_groups = slots.groupby("spot_id")
# slot_groups 的 key 是 spot_id (1-indexed)

```

```

for j in range(n_users):
    a, d = users.iloc[j]["arrival"], users.iloc[j]["departure"]
    # 只需检查 eligible 的 spot
    candidate_spots = np.where(eligible[j])[0] # 0-indexed
    for idx_i in candidate_spots:
        spot_id = idx_i + 1 # spot_id 是 1-indexed (来自CSV)
        if spot_id not in slot_groups.groups:
            continue
        group = slot_groups.get_group(spot_id)
        for _, row in group.iterrows():
            if row["start"] <= a and d <= row["end"]:
                time_eligible[j, idx_i] = True
                break

```

```
return time_eligible
```

```
def check_time_conflict(users, user_pairs=None):
```

```

"""
检查用户之间是否有时间冲突 (区间重叠)
返回: conflict[j, k] = True if 用户j和用户k时间区间重叠

```

```

使用半开区间 [arrival, departure) 约定:
重叠判定: [a1, d1) ∩ [a2, d2) ≠ ∅ ⇔ a1 < d2 AND a2 < d1

```

```
使用用户0-indexed ID
```

```

"""
n = len(users)
a = users["arrival"].to_numpy() # (n,)
d = users["departure"].to_numpy() # (n,)
# 正确向量化: [a1, d1) 和 [a2, d2) 重叠 iff a1 < d2 AND a2 < d1
conflict = (a[:, None] < d[None, :]) & (a[None, :] < d[:, None]) # (n, n)
np.fill_diagonal(conflict, False)
return conflict

```

```

def find_spot_available_slots(users, slots):
    """
    对每个车位，构建其可用时段列表
    返回: {spot_id: [(start, end), ...]}
    """
    available = {}
    for spot_id, group in slots.groupby("spot_id"):
        available[spot_id] = list(zip(group["start"], group["end"]))
    return available

def find_candidate_users_for_spot(users, slots, eligible):
    """
    对每个车位 i，找到兼容的用户列表（距离 + 时间兼容）
    返回: candidates[i] = [j1, j2, ...]
    """
    n_users = len(users)
    n_spots = eligible.shape[1]
    slot_groups = slots.groupby("spot_id")

    candidates = [[] for _ in range(n_spots)]

    for i in range(n_spots):
        spot_id = i + 1
        if spot_id not in slot_groups.groups:
            continue
        time_windows = list(zip(
            slot_groups.get_group(spot_id)["start"],
            slot_groups.get_group(spot_id)["end"]
        ))

        for j in range(n_users):
            if not eligible[j, i]:
                continue
            a, d = users.iloc[j]["arrival"], users.iloc[j]["departure"]
            for s, e in time_windows:
                if s <= a and d <= e:
                    candidates[i].append(j)
                    break

    return candidates

```

plot_theme.py — Matplotlib 自定义主题

```

import matplotlib
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap

def _make_cmap():
    # Professional, print-friendly blue→cyan→green→amber gradient
    return LinearSegmentedColormap.from_list(

```

```

        "geo_blue_grad",
        [
            (0.00, "#0B1F5C"), # deep navy
            (0.35, "#1EA7FD"), # sky blue
            (0.62, "#00C389"), # teal/green
            (1.00, "#F6C453"), # amber
        ],
    )

CMAP = _make_cmap()

def _rgba_to_hex(rgba):
    r, g, b, a = rgba
    return "#{:02X}{:02X}{:02X}".format(int(r * 255), int(g * 255), int(b * 255))

def pick_color(t: float) -> str:
    """Pick a color from the shared gradient (t in [0,1])."""
    t = float(max(0.0, min(1.0, t)))
    return _rgba_to_hex(CMAP(t))

def apply_common_style():
    # Fonts (fallbacks for Windows)
    plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei", "SimSun"]
    plt.rcParams["axes.unicode_minus"] = False

    # Clean, paper-friendly look
    plt.rcParams["figure.facecolor"] = "white"
    plt.rcParams["axes.facecolor"] = "white"
    plt.rcParams["axes.edgecolor"] = "#B0B0B0"
    plt.rcParams["axes.linewidth"] = 0.9

    # Grid: subtle
    plt.rcParams["grid.color"] = "#E6E6E6"
    plt.rcParams["grid.linewidth"] = 0.8
    plt.rcParams["grid.alpha"] = 1.0
    plt.rcParams["grid.linestyle"] = "-"

    # Lines
    plt.rcParams["lines.linewidth"] = 2.0

    # Ensure SVG/PNG anti-aliasing behaves well
    matplotlib.rcParams["savefig.facecolor"] = "white"

```

visual_helpers.py — SVG 热力图辅助

```

import numpy as np
import matplotlib.pyplot as plt
from matplotlib import cm

```

```

def save_svg_heatmap(data, x_coords, y_coords, outpath):
    """Save a simple SVG heatmap for inclusion in LaTeX."""
    import svgwrite
    dwg = svgwrite.Drawing(outpath, size=(800, 400))
    dwg.add(dwg.rect(insert=(0,0), size=('100%', '100%'), fill='white'))
    # simple grid
    nx = len(x_coords)
    ny = len(y_coords)
    cell_w = 700 / nx
    cell_h = 320 / ny
    # normalize
    vmin = np.nanmin(data)
    vmax = np.nanmax(data)
    for i in range(nx):
        for j in range(ny):
            v = data[i, j]
            if not np.isfinite(v):
                color = '#f0f0f0'
            else:
                frac = (v - vmin) / (vmax - vmin + 1e-9)
                # use viridis-like gradient
                cmap = cm.get_cmap('viridis')
                rgba = cmap(frac)
                color = svgwrite.utils.rgb(int(rgba[0]*255), int(rgba[1]*255),
int(rgba[2]*255))
                x = 50 + i * cell_w
                y = 50 + (ny-1-j) * cell_h
                dwg.add(dwg.rect(insert=(x,y), size=(cell_w, cell_h), fill=color,
stroke='none'))
            dwg.save()

```

gen_svgs.py — 生成热力图输出

```

"""Generate SVG slices and 3D renders for selected hours using q4_heatmap
utilities."""
from q4_heatmap import load_data, build_spatial_temporal_pressure, save_svg_slice,
plot_heatmap_3d
import os

def main():
    spots, slots, users = load_data()
    radius = users['max_walk'].median()
    grid_x, grid_y, times, demand_grid, supply_grid, reachable_grid, pressure_grid
= \
        build_spatial_temporal_pressure(spots, slots, users, grid_size=10,
radius=radius)

    # select indices for 12 and 13
    idx_12 = min(range(len(times)), key=lambda i: abs(times[i]-12))
    idx_13 = min(range(len(times)), key=lambda i: abs(times[i]-13))

```

```

outdir = 'svg_out'
os.makedirs(outdir, exist_ok=True)

save_svg_slice(pressure_grid[:, :, idx_12].copy(), grid_x, grid_y,
os.path.join(outdir, 'heatmap_12h.svg'), 12)
save_svg_slice(pressure_grid[:, :, idx_13].copy(), grid_x, grid_y,
os.path.join(outdir, 'heatmap_13h.svg'), 13)

plot_heatmap_3d(grid_x, grid_y, times, pressure_grid, t_idx=idx_12,
outpath=os.path.join(outdir, 'heatmap_12h_3d.png'))
plot_heatmap_3d(grid_x, grid_y, times, pressure_grid, t_idx=idx_13,
outpath=os.path.join(outdir, 'heatmap_13h_3d.png'))

print('Done: SVGs and 3D renders saved to', outdir)

if __name__ == '__main__':
    main()

```

analyze_units.py — 数据探索工具

```

import pandas as pd
import numpy as np
import math
import sys

def read_csv_gbk(filepath):
    """Read CSV file with GBK encoding"""
    try:
        df = pd.read_csv(filepath, encoding='gbk')
        return df
    except:
        # Try other encodings if GBK fails
        try:
            df = pd.read_csv(filepath, encoding='utf-8')
            return df
        except:
            df = pd.read_csv(filepath, encoding='latin1')
            return df

def main():
    # Read data files
    users_path = r'C:\Users\John\Desktop\国赛校\重庆大学2026数学建模春季赛题目\B题附件\users.csv'
    spots_path = r'C:\Users\John\Desktop\国赛校\重庆大学2026数学建模春季赛题目\B题附件\spots.csv'

    print("Reading users.csv...")
    users_df = read_csv_gbk(users_path)
    print(f"Users shape: {users_df.shape}")
    print(f"Users columns: {list(users_df.columns)}")

    print("\nReading spots.csv...")

```

```

spots_df = read_csv_gbk(spots_path)
print(f"Spots shape: {spots_df.shape}")
print(f"Spots columns: {list(spots_df.columns)}")

# Rename columns for clarity (based on garbled Chinese headers)
# users.csv: 用户ID, 目的地X, 目的地Y, 最大步行距离, 到达时间, 离开时间
# spots.csv: 车位ID, X坐标, Y坐标

if len(users_df.columns) == 6:
    users_df.columns = ['user_id', 'dest_x', 'dest_y', 'max_walk_dist',
'arrival_time', 'departure_time']
else:
    # If column names are garbled, assume first 6 columns
    users_df = users_df.iloc[:, :6]
    users_df.columns = ['user_id', 'dest_x', 'dest_y', 'max_walk_dist',
'arrival_time', 'departure_time']

if len(spots_df.columns) == 3:
    spots_df.columns = ['spot_id', 'spot_x', 'spot_y']
else:
    # If column names are garbled, assume first 3 columns
    spots_df = spots_df.iloc[:, :3]
    spots_df.columns = ['spot_id', 'spot_x', 'spot_y']

# Convert to numeric
users_df['dest_x'] = pd.to_numeric(users_df['dest_x'], errors='coerce')
users_df['dest_y'] = pd.to_numeric(users_df['dest_y'], errors='coerce')
users_df['max_walk_dist'] = pd.to_numeric(users_df['max_walk_dist'],
errors='coerce')

spots_df['spot_x'] = pd.to_numeric(spots_df['spot_x'], errors='coerce')
spots_df['spot_y'] = pd.to_numeric(spots_df['spot_y'], errors='coerce')

# Basic statistics
print("\n=== USER COORDINATE STATISTICS ===")
print(f"Number of users: {len(users_df)}")
print(f"Destination X - Min: {users_df['dest_x'].min():.2f}, Max:
{users_df['dest_x'].max():.2f}, Mean: {users_df['dest_x'].mean():.2f}, Std:
{users_df['dest_x'].std():.2f}")
print(f"Destination Y - Min: {users_df['dest_y'].min():.2f}, Max:
{users_df['dest_y'].max():.2f}, Mean: {users_df['dest_y'].mean():.2f}, Std:
{users_df['dest_y'].std():.2f}")
print(f"Max walk distance - Min: {users_df['max_walk_dist'].min():.2f}, Max:
{users_df['max_walk_dist'].max():.2f}, Mean:
{users_df['max_walk_dist'].mean():.2f}, Std:
{users_df['max_walk_dist'].std():.2f}")

print("\n=== SPOT COORDINATE STATISTICS ===")
print(f"Number of spots: {len(spots_df)}")
print(f"Spot X - Min: {spots_df['spot_x'].min():.2f}, Max:
{spots_df['spot_x'].max():.2f}, Mean: {spots_df['spot_x'].mean():.2f}, Std:
{spots_df['spot_x'].std():.2f}")
print(f"Spot Y - Min: {spots_df['spot_y'].min():.2f}, Max:
{spots_df['spot_y'].max():.2f}, Mean: {spots_df['spot_y'].mean():.2f}, Std:
{spots_df['spot_y'].std():.2f}")

# Compute Euclidean distances for all user-spot pairs

```

```

print("\n=== COMPUTING ALL USER-SPOT DISTANCES ===")
n_users = len(users_df)
n_spots = len(spots_df)
total_pairs = n_users * n_spots
print(f"Computing {total_pairs:,} distances ({n_users} users x {n_spots}
spots)...")

# Use vectorized computation for efficiency
distances = []

# For large datasets, process in batches
batch_size = 1000
for i in range(0, n_users, batch_size):
    user_batch = users_df.iloc[i:i+batch_size]
    user_x = user_batch['dest_x'].values
    user_y = user_batch['dest_y'].values

    # Compute distances to all spots for this batch
    for j in range(n_spots):
        spot_x = spots_df.iloc[j]['spot_x']
        spot_y = spots_df.iloc[j]['spot_y']
        batch_dist = np.sqrt((user_x - spot_x)**2 + (user_y - spot_y)**2)
        distances.extend(batch_dist)

    if (i // batch_size) % 10 == 0:
        print(f"  Processed {min(i+batch_size, n_users)}/{n_users} users...")

distances = np.array(distances)

print(f"\n=== DISTANCE DISTRIBUTION ===")
print(f"Total pairs: {len(distances):,}")
print(f"Min distance: {distances.min():.4f}")
print(f"Max distance: {distances.max():.4f}")
print(f"Mean distance: {distances.mean():.4f}")
print(f"Median distance: {np.median(distances):.4f}")
print(f"Std deviation: {distances.std():.4f}")

# Percentiles
percentiles = [1, 5, 10, 25, 50, 75, 90, 95, 99]
for p in percentiles:
    print(f"{p}th percentile: {np.percentile(distances, p):.4f}")

# Count pairs with distance <= 15.33 (claimed average walk threshold w_j)
threshold = 15.33
within_threshold = np.sum(distances <= threshold)
percentage = (within_threshold / len(distances)) * 100
print(f"\nPairs with distance <= {threshold}: {within_threshold:,}
({percentage:.2f}%)")

# Theoretical average distance in a 100x100 unit square
# For uniformly distributed points in [0,100]x[0,100]
# Expected Euclidean distance ≈ 52.34 (theoretical value for unit square
[0,1]x[0,1] is ~0.5214, scaled by 100)
theoretical_avg = 0.5214 * 100 # For unit square scaled by 100
print(f"\n=== THEORETICAL COMPARISON ===")
print(f"Theoretical average distance in [0,100]x[0,100] unit square: ~
{theoretical_avg:.2f}")

```

```

print(f"Actual average distance: {distances.mean():.2f}")
print(f"Ratio (actual/theoretical): {distances.mean()/theoretical_avg:.3f}")

# Check if coordinates could be in meters
print(f"\n=== PHYSICAL INTERPRETATION ===")
print(f"If coordinates are in meters in a 100m×100m area:")
print(f" - Average walking distance would be {distances.mean():.2f} meters")
print(f" - This is {distances.mean():.2f} meters ≈ {distances.mean()/100:.2f} km")
print(f" - Paper claims average walking distance of 2.82 units")
print(f" - Ratio: 2.82 / {distances.mean():.2f} = {2.82/distances.mean():.4f}")

# Alternative: if coordinates are in 100m units (hectometers)
print(f"\nIf coordinates are in 100m units (hectometers):")
print(f" - Actual distance in meters = {distances.mean() * 100:.2f} meters")
print(f" - Ratio to paper's 2.82: {distances.mean() * 100 / 2.82:.2f}")

# Alternative: if coordinates are in km
print(f"\nIf coordinates are in km:")
print(f" - Actual distance in meters = {distances.mean() * 1000:.2f} meters")
print(f" - Ratio to paper's 2.82: {distances.mean() * 1000 / 2.82:.2f}")

return users_df, spots_df, distances

if __name__ == "__main__":
    users_df, spots_df, distances = main()

```

问题1：静态分配 `q1_static/`

`q1_static.py`

"""问题1：静态分配（无时间约束）

两阶段优化：

阶段1：最大化匹配用户数（最大基数匹配）

阶段2：在最大匹配数下最小化总步行距离

模型：二分图分配，每个车位最多一个用户，每个用户最多一个车位

"""

```

import pulp
import numpy as np
from utils import load_data, compute_distance_matrix, build_eligibility

```

```

def solve_q1(verbose=True):
    """求解问题1，返回分配结果"""
    spots, slots, users = load_data()

```

```

dist_matrix = compute_distance_matrix(spots, users)
eligible = build_eligibility(dist_matrix, users)

n_users, n_spots = eligible.shape
user_ids = users["user_id"].tolist()
spot_ids = spots["spot_id"].tolist()

# 构建可用边列表 (user_idx, spot_idx)
edges = []
for j in range(n_users):
    for i in range(n_spots):
        if eligible[j, i]:
            edges.append((j, i))

if not edges:
    return {"assigned": [], "unassigned": user_ids, "n_assigned": 0,
            "avg_distance": 0, "utilization": 0, "status": "no_eligible"}

# ===== 阶段1: 最大化匹配用户数 =====
if verbose:
    print("=" * 60)
    print("问题1: 静态分配（无时间约束）")
    print("=" * 60)
    print(f"可用边数: {len(edges)} / {n_users * n_spots}")
    print("\n阶段1: 求解最大基数匹配...")

prob1 = pulp.LpProblem("Q1_MaxCardinality", pulp.LpMaximize)
x_vars = {}
for (j, i) in edges:
    x_vars[(j, i)] = pulp.LpVariable(f"x_{j}_{i}", cat="Binary")

# 目标: 最大化匹配用户数
prob1 += pulp.lpSum([x_vars[e] for e in edges])

# 约束: 每个用户最多一个车位
for j in range(n_users):
    incident = [(j, i) for (uj, i) in edges if uj == j]
    if incident:
        prob1 += pulp.lpSum([x_vars[e] for e in incident]) <= 1

# 约束: 每个车位最多一个用户
for i in range(n_spots):
    incident = [(j, i) for (j, si) in edges if si == i]
    if incident:
        prob1 += pulp.lpSum([x_vars[e] for e in incident]) <= 1

# 求解
prob1.solve(pulp.PULP_CBC_CMD(msg=verbose))

if prob1.status != pulp.LpStatusOptimal:
    if verbose:
        print(f"阶段1求解失败, 状态: {pulp.LpStatus[prob1.status]}")
    return {"assigned": [], "unassigned": user_ids, "n_assigned": 0,
            "avg_distance": 0, "utilization": 0, "status": "phase1_failed"}

# 获取最大匹配数
max_assigned = int(pulp.value(prob1.objective))

```

```

if verbose:
    print(f"最大匹配用户数: {max_assigned}")

# ===== 阶段2: 固定匹配数, 最小化总步行距离 =====
if verbose:
    print("\n阶段2: 最小化总步行距离...")

prob2 = pulp.LpProblem("Q1_MinDistance", pulp.LpMinimize)
x_vars2 = {}
for (j, i) in edges:
    x_vars2[(j, i)] = pulp.LpVariable(f"x2_{j}_{i}", cat="Binary")

# 目标: 最小化总步行距离
prob2 += pulp.lpSum([dist_matrix[j, i] * x_vars2[(j, i)] for (j, i) in edges])

# 约束1: 每个用户最多一个车位
for j in range(n_users):
    incident = [(j, i) for (uj, i) in edges if uj == j]
    if incident:
        prob2 += pulp.lpSum([x_vars2[e] for e in incident]) <= 1

# 约束2: 每个车位最多一个用户
for i in range(n_spots):
    incident = [(j, i) for (j, si) in edges if si == i]
    if incident:
        prob2 += pulp.lpSum([x_vars2[e] for e in incident]) <= 1

# 约束3: 总匹配数 = 最大匹配数
prob2 += pulp.lpSum([x_vars2[e] for e in edges]) == max_assigned

# 求解
prob2.solve(pulp.PULP_CBC_CMD(msg=verbose))

if prob2.status != pulp.LpStatusOptimal:
    if verbose:
        print(f"阶段2求解失败, 状态: {pulp.LpStatus[prob2.status]}")
    return {"assigned": [], "unassigned": user_ids, "n_assigned": 0,
            "avg_distance": 0, "utilization": 0, "status": "phase2_failed"}

# ===== 提取结果 =====
assignment = {} # spot_idx -> user_idx
for (j, i) in edges:
    if pulp.value(x_vars2[(j, i)]) > 0.5:
        assignment[i] = j

assigned_users = sorted([user_ids[assignment[i]] for i in assignment])
unassigned_users = sorted(set(user_ids) - set(assigned_users))
n_assigned = len(assigned_users)
total_distance = sum(dist_matrix[assignment[i], i] for i in assignment)
avg_distance = total_distance / n_assigned if n_assigned > 0 else 0
utilization = n_assigned / n_spots

# 构建详细分配表
assignment_detail = []
for spot_idx, user_idx in assignment.items():
    assignment_detail.append({
        "spot_id": spot_ids[spot_idx],

```

```

        "user_id": user_ids[user_idx],
        "distance": dist_matrix[user_idx, spot_idx]
    })

result = {
    "assigned": assigned_users,
    "unassigned": unassigned_users,
    "n_assigned": n_assigned,
    "total_distance": total_distance,
    "avg_distance": avg_distance,
    "utilization": utilization,
    "assignment_detail": assignment_detail,
    "status": "optimal"
}

if verbose:
    print(f"\n===== 问题1 结果 =====")
    print(f"被分配用户数: {n_assigned}")
    print(f"总步行距离: {total_distance:.2f} 米")
    print(f"平均步行距离: {avg_distance:.2f} 米")
    print(f"车位利用率: {utilization:.2%} ({n_assigned}/{n_spots})")
    print(f"未分配用户数: {len(unassigned_users)}")

return result

if __name__ == "__main__":
    result = solve_q1(verbose=True)

```

问题2：时间约束分配 `q2_time/`

`q2_time.py`

"""问题2：时间约束分配

考虑车位的可用时段和用户的时间窗口。
 每个车位可服务多个用户，只要他们的时间窗口不重叠。
 用户只能使用完全覆盖其[到达, 离开]时段的车位时段。

模型：MILP

- `x_ij`: 用户`j`分配给车位`i`
- 每个用户最多一个车位
- 每个车位同一时间只能服务一个用户 → 冲突用户对的互斥约束
- 两级目标：最大化服务用户数，再最小化总步行距离

"""

```

import pulp
import numpy as np
from utils import load_data, compute_distance_matrix, build_eligibility,

```

build_time_eligibility

```
def build_spot_time_conflicts(users, slots, eligible):
    """
    对每个车位i，构建内部用户时间冲突列表。
    返回: conflict_map[i] = [(j1, j2), ...] 表示在车位i上j1和j2不能同时被分配
    """

    n_users = len(users)
    n_spots = eligible.shape[1]
    slot_groups = slots.groupby("spot_id")

    conflict_map = [[] for _ in range(n_spots)]
    intervals = users[["arrival", "departure"]].values

    for i in range(n_spots):
        spot_id = i + 1
        if spot_id not in slot_groups.groups:
            continue
        time_windows = list(zip(
            slot_groups.get_group(spot_id)["start"],
            slot_groups.get_group(spot_id)["end"]
        ))

        # 找到对该车位兼容的用户列表
        candidate_users = []
        for j in range(n_users):
            if not eligible[j, i]:
                continue
            a, d = intervals[j, 0], intervals[j, 1]
            for s, e in time_windows:
                if s <= a and d <= e:
                    candidate_users.append(j)
                    break

        # 检查两两时间冲突
        for idx1 in range(len(candidate_users)):
            for idx2 in range(idx1 + 1, len(candidate_users)):
                j1 = candidate_users[idx1]
                j2 = candidate_users[idx2]
                a1, d1 = intervals[j1, 0], intervals[j1, 1]
                a2, d2 = intervals[j2, 0], intervals[j2, 1]
                if a1 < d2 and a2 < d1:
                    conflict_map[i].append((j1, j2))

    return conflict_map


def solve_q2(verbose=True):
    """求解问题2，返回分配结果"""
    spots, slots, users = load_data()
    dist_matrix = compute_distance_matrix(spots, users)
    eligible = build_eligibility(dist_matrix, users)
    time_eligible = build_time_eligibility(users, slots, eligible)

    n_users, n_spots = time_eligible.shape
    user_ids = users["user_id"].tolist()
```

```

spot_ids = spots["spot_id"].tolist()

# 构建可用边列表（距离+时间都兼容）
edges = []
for j in range(n_users):
    for i in range(n_spots):
        if time_eligible[j, i]:
            edges.append((j, i))

if not edges:
    return {"assigned": [], "unassigned": user_ids, "n_assigned": 0,
           "avg_distance": 0, "utilization": 0, "status": "no_eligible"}

if verbose:
    print("=" * 60)
    print("问题2: 时间约束分配")
    print("=" * 60)
    print(f"距离+时间兼容边数: {len(edges)}")

conflict_map = build_spot_time_conflicts(users, slots, time_eligible)
total_conflicts = sum(len(c) for c in conflict_map)
if verbose:
    print(f"车位内部时间冲突对数: {total_conflicts}")

# ===== 阶段1: 最大化匹配用户数 =====
if verbose:
    print("\n阶段1: 求解最大基数匹配（带时间约束）...")

prob1 = pulp.LpProblem("Q2_MaxCardinality", pulp.LpMaximize)
x_vars = {}
for (j, i) in edges:
    x_vars[(j, i)] = pulp.LpVariable(f"x_{j}_{i}", cat="Binary")

prob1 += pulp.lpSum([x_vars[e] for e in edges])

for j in range(n_users):
    incident = [(j, i) for (uj, i) in edges if uj == j]
    if incident:
        prob1 += pulp.lpSum([x_vars[e] for e in incident]) <= 1

for i in range(n_spots):
    for (j1, j2) in conflict_map[i]:
        if (j1, i) in x_vars and (j2, i) in x_vars:
            prob1 += x_vars[(j1, i)] + x_vars[(j2, i)] <= 1

prob1.solve(pulp.PULP_CBC_CMD(msg=verbose))

if prob1.status != pulp.LpStatusOptimal:
    if verbose:
        print(f"阶段1求解失败: {pulp.LpStatus[prob1.status]}")
    return {"assigned": [], "unassigned": user_ids, "n_assigned": 0,
           "avg_distance": 0, "utilization": 0, "status": "phase1_failed"}

max_assigned = int(pulp.value(prob1.objective))
if verbose:
    print(f"最大匹配用户数: {max_assigned}")

```

```

# ===== 阶段2: 固定匹配数, 最小化总步行距离 =====
if verbose:
    print("\n阶段2: 最小化总步行距离...")

prob2 = pulp.LpProblem("Q2_MinDistance", pulp.LpMinimize)
x_vars2 = {}
for (j, i) in edges:
    x_vars2[(j, i)] = pulp.LpVariable(f"x2_{j}_{i}", cat="Binary")

prob2 += pulp.lpSum([dist_matrix[j, i] * x_vars2[(j, i)] for (j, i) in edges])

for j in range(n_users):
    incident = [(j, i) for (uj, i) in edges if uj == j]
    if incident:
        prob2 += pulp.lpSum([x_vars2[e] for e in incident]) <= 1

for i in range(n_spots):
    for (j1, j2) in conflict_map[i]:
        if (j1, i) in x_vars2 and (j2, i) in x_vars2:
            prob2 += x_vars2[(j1, i)] + x_vars2[(j2, i)] <= 1

prob2 += pulp.lpSum([x_vars2[e] for e in edges]) == max_assigned

prob2.solve(pulp.PULP_CBC_CMD(msg=verbose))

if prob2.status != pulp.LpStatusOptimal:
    if verbose:
        print(f"阶段2求解失败: {pulp.LpStatus[prob2.status]}")
    return {"assigned": [], "unassigned": user_ids, "n_assigned": 0,
            "avg_distance": 0, "utilization": 0, "status": "phase2_failed"}

# ===== 提取结果 =====
assignment = {}
for (j, i) in edges:
    if pulp.value(x_vars2[(j, i)]) > 0.5:
        if i not in assignment:
            assignment[i] = []
        assignment[i].append(j)

# 统计所有被分配的用户
assigned_set = set()
assignment_detail = []
for spot_idx, user_idx in assignment.items():
    for user_idx in user_idx:
        assigned_set.add(user_idx)
        assignment_detail.append({
            "spot_id": spot_ids[spot_idx],
            "user_id": user_ids[user_idx],
            "distance": float(dist_matrix[user_idx, spot_idx])
        })

n_assigned = len(assigned_set)
total_distance = sum(
    dist_matrix[user_idx, spot_idx]
    for spot_idx, user_idx in assignment.items()
    for user_idx in user_idx
)

```

```

avg_distance = total_distance / n_assigned if n_assigned > 0 else 0
utilization = n_assigned / n_spots

assigned_users = sorted([user_ids[u] for u in assigned_set])
unassigned_users = sorted(set(user_ids) - set(assigned_users))

result = {
    "assigned": assigned_users,
    "unassigned": unassigned_users,
    "n_assigned": n_assigned,
    "total_distance": total_distance,
    "avg_distance": avg_distance,
    "utilization": utilization,
    "assignment_detail": assignment_detail,
    "status": "optimal"
}

if verbose:
    print(f"\n===== 问题2 结果 =====")
    print(f"被分配用户数: {n_assigned}")
    print(f"总步行距离: {total_distance:.2f} 米")
    print(f"平均步行距离: {avg_distance:.2f} 米")
    print(f"车位利用率: {utilization:.2%} ({n_assigned}/{n_spots})")
    print(f"未分配用户数: {len(unassigned_users)}")

return result

if __name__ == "__main__":
    result = solve_q2(verbose=True)

```

问题3：在线调度 `q3_online/`

`q3_online.py`

"""问题3：在线调度算法

用户按到达时间顺序出现，平台需立即做出分配决策（不可更改）。
目标是最大化服务用户数，同时最小化步行距离。

算法策略：

1. **Greedy-Nearest**：选择离目的地最近的可用车位
2. **Greedy-Balance**：选择兼容且当前利用率最低的车位
3. **竞争比分析**：与问题2离线最优解比较

"""

```

import time
import numpy as np
from utils import load_data, compute_distance_matrix, build_eligibility

```

```

class OnlineAllocator:
    """在线分配器，管理车位状态和分配决策"""

    def __init__(self, spots, slots, users, dist_matrix, eligible):
        self.spots = spots
        self.slots = slots
        self.users = users
        self.dist_matrix = dist_matrix
        self.eligible = eligible

        self.n_spots = len(spots)
        self.n_users = len(users)

        # spot -> list of (slot_start, slot_end) 每个车位可用时段
        self.spot_slots = {}
        for spot_id, group in slots.groupby("spot_id"):
            self.spot_slots[spot_id] = list(zip(group["start"], group["end"]))

        # spot -> list of (user_start, user_end, user_id) 已分配的占用区间
        self.occupancy = {spot_id: [] for spot_id in spots["spot_id"]}

        # 统计
        self.assigned = []
        self.rejected = []
        self.total_distance = 0.0

    def is_time_compatible(self, spot_id, user_arrival, user_departure):
        """检查用户时间窗口是否在车位的某个可用时段内"""
        if spot_id not in self.spot_slots:
            return False
        for s, e in self.spot_slots[spot_id]:
            if s <= user_arrival and user_departure <= e:
                return True
        return False

    def is_spot_free(self, spot_id, user_arrival, user_departure):
        """检查车位在用户时间段内是否空闲"""
        if spot_id not in self.occupancy:
            return True
        for (occ_start, occ_end, _) in self.occupancy[spot_id]:
            if user_arrival < occ_end and occ_start < user_departure:
                return False
        return True

    def assign_user(self, user_idx, spot_idx):
        """分配用户到车位"""
        spot_id = self.spots.iloc[spot_idx]["spot_id"]
        user_id = self.users.iloc[user_idx]["user_id"]
        arrival = self.users.iloc[user_idx]["arrival"]
        departure = self.users.iloc[user_idx]["departure"]
        distance = self.dist_matrix[user_idx, spot_idx]

        self.occupancy[spot_id].append((arrival, departure, user_id))
        self.assigned.append({
            "user_id": user_id,

```

```

        "spot_id": spot_id,
        "distance": distance,
        "arrival": arrival,
        "departure": departure
    })
    self.total_distance += distance
    return True

```

```

def find_candidates(self, user_idx):
    """找到用户的所有候选车位（距离+时间兼容且空闲）"""
    arrival = self.users.iloc[user_idx]["arrival"]
    departure = self.users.iloc[user_idx]["departure"]
    candidates = []

    for i in range(self.n_spots):
        if not self.eligible[user_idx, i]:
            continue
        spot_id = self.spots.iloc[i]["spot_id"]
        if not self.is_time_compatible(spot_id, arrival, departure):
            continue
        if not self.is_spot_free(spot_id, arrival, departure):
            continue
        candidates.append(i)

    return candidates

def greedy_nearest(self, user_idx):
    """策略1：选择最近的车位"""
    candidates = self.find_candidates(user_idx)
    if not candidates:
        return False

    # 选最近
    best_i = min(candidates, key=lambda i: self.dist_matrix[user_idx, i])
    return self.assign_user(user_idx, best_i)

```

```

def greedy_balance(self, user_idx):
    """策略2：选择利用率最低的车位"""
    candidates = self.find_candidates(user_idx)
    if not candidates:
        return False

    # 选当前占用最少（利用率低）的车位
    def spot_load(i):
        spot_id = self.spots.iloc[i]["spot_id"]
        return len(self.occupancy[spot_id])

    best_i = min(candidates, key=lambda i: (
        spot_load(i),
        self.dist_matrix[user_idx, i]
    ))
    return self.assign_user(user_idx, best_i)

```

```

def solve_q3(strategy="nearest", verbose=True):
    """运行在线调度模拟"""
    spots, slots, users = load_data()

```

```

dist_matrix = compute_distance_matrix(spots, users)
eligible = build_eligibility(dist_matrix, users)

# 按到达时间排序
sorted_users = users.sort_values("arrival")
# 相同时刻按用户ID排序
sorted_users = sorted_users.sort_values(["arrival", "user_id"])

allocator = OnlineAllocator(spots, slots, users, dist_matrix, eligible)

t_start = time.time()

if verbose:
    print("=" * 60)
    print(f"问题3: 在线调度算法 (策略: {strategy}) ")
    print("=" * 60)

for idx, (_, user_row) in enumerate(sorted_users.iterrows()):
    user_idx = user_row.name # 原始索引
    if strategy == "nearest":
        accepted = allocator.greedy_nearest(user_idx)
    elif strategy == "balance":
        accepted = allocator.greedy_balance(user_idx)
    else:
        accepted = allocator.greedy_nearest(user_idx)

    if not accepted:
        allocator.rejected.append(user_idx)

    if verbose and (idx + 1) % 100 == 0:
        print(f" 已处理 {idx + 1}/{len(sorted_users)} 用户...")

elapsed = time.time() - t_start

n_assigned = len(allocator.assigned)
avg_distance = allocator.total_distance / n_assigned if n_assigned > 0 else 0
utilization = n_assigned / allocator.n_spots

result = {
    "strategy": strategy,
    "assigned": [a["user_id"] for a in allocator.assigned],
    "n_assigned": n_assigned,
    "total_distance": allocator.total_distance,
    "avg_distance": avg_distance,
    "utilization": utilization,
    "assignment_detail": allocator.assigned,
    "elapsed_seconds": elapsed,
    "status": "ok"
}

if verbose:
    print(f"\n===== 问题3 结果 =====")
    print(f"策略: {strategy}")
    print(f"被分配用户数: {n_assigned}")
    print(f"总步行距离: {allocator.total_distance:.2f} 米")
    print(f"平均步行距离: {avg_distance:.2f} 米")
    print(f"车位利用率: {utilization:.2%} ({n_assigned}/{allocator.n_spots})")

```

```

print(f"拒绝用户数: {len(allocator.rejected)}")
print(f"运行时间: {elapsed:.4f} 秒")

return result

def compare_with_offline(online_result, offline_result, verbose=True):
    """比较在线与离线结果，计算竞争比"""
    if offline_result["status"] != "optimal" or online_result["status"] != "ok":
        if verbose:
            print("无法比较: 结果不完整")
        return

    online_n = online_result["n_assigned"]
    offline_n = offline_result["n_assigned"]

    # 竞争比（服务用户数）
    ratio_users = online_n / offline_n if offline_n > 0 else 0

    # 平均距离比
    online_avg = online_result["avg_distance"]
    offline_avg = offline_result["avg_distance"]
    ratio_dist = online_avg / offline_avg if offline_avg > 0 else float("inf")

    if verbose:
        print(f"\n===== 竞争比分析 =====")
        print(f"{'指标':<25} {'在线':<12} {'离线':<12} {'比值':<12}")
        print(f"{'='*61}")
        print(f"{'服务用户数':<25} {online_n:<12} {offline_n:<12} {ratio_users:
<12.4f}")
        print(f"{'平均步行距离(米)':<25} {online_avg:<12.2f} {offline_avg:<12.2f}
{ratio_dist:<12.4f}")
        print(f"\n竞争比（服务用户数）: {ratio_users:.4f}")
        print(f" 含义: 在线算法达到了离线最优的 {ratio_users:.2%}")

    return {
        "ratio_users": ratio_users,
        "ratio_distance": ratio_dist,
        "online_n": online_n,
        "offline_n": offline_n,
        "online_avg_dist": online_avg,
        "offline_avg_dist": offline_avg
    }

if __name__ == "__main__":
    result = solve_q3(strategy="nearest", verbose=True)

```

问题4：供需分析 [q4_analysis/](#)

[q4_analysis.py](#) — 供需匹配分析

"""问题4：供需匹配分析与建议

分析内容：

1. 时段供需分析 - 哪些时段车位最紧张
2. 步行距离分布
3. 空间热力图
4. 运营建议

"""

```
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg") # 非交互式后端
import matplotlib.pyplot as plt

from plot_theme import apply_common_style, pick_color

apply_common_style()
import sys

# 控制台 UTF-8 编码，确保中文正常输出
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")

from utils import load_data, compute_distance_matrix, build_eligibility,
build_time_eligibility

from q1_static import solve_q1
from q2_time import solve_q2
from q3_online import solve_q3

def analyze_supply_demand(spots, slots, users, verbose=True):
    """分析各时段供需情况"""
    # 时间范围：8-22 小时
    time_range = np.arange(8, 22.5, 0.5) # 每半小时一个点

    n_slots = len(time_range)
    supply = np.zeros(n_slots) # 每个时间点可用车位数
    demand = np.zeros(n_slots) # 每个时间点活跃用户数

    # 计算每个时间点的供给（半开区间 [start, end)）
    for _, row in slots.iterrows():
        s, e = row["start"], row["end"]
        for t_idx, t in enumerate(time_range):
            if s <= t < e:
                supply[t_idx] += 1

    # 计算每个时间点的需求（半开区间 [arrival, departure)）
    for _, row in users.iterrows():
        a, d = row["arrival"], row["departure"]
        for t_idx, t in enumerate(time_range):
            if a <= t < d:
                demand[t_idx] += 1

    supply_demand = pd.DataFrame({
```

```

        "time": time_range,
        "supply": supply,
        "demand": demand,
        "gap": supply - demand,
        "pressure": np.where(supply > 0, demand / supply, np.inf)
    })

    if verbose:
        print("\n===== 时段供需分析 =====")
        print(f"{'时段':<10} {'供给(车位)':<14} {'需求(用户)':<14} {'缺口':<10} {'压力比':<10}")
        print("-" * 58)
        for _, row in supply_demand.iterrows():
            hour = row["time"]
            pressure_str = f"{row['pressure']:.2f}" if np.isfinite(row["pressure"])
        else "∞"
            if row["gap"] < 0:
                print(f"{hour:<8.1f} {row['supply']:<14.0f} {row['demand']:<14.0f} {row['gap']:<10.0f} {pressure_str:<10}")
                print("-" * 58)

            # 找出最紧张的时段（过滤掉供给为0的时段，避免inf干扰）
            supply_demand_valid = supply_demand[(supply_demand["demand"] > 0) &
            (supply_demand["supply"] > 0)].copy()
            tightest =
            supply_demand_valid.loc[supply_demand_valid["pressure"].idxmax()]
            print(f"\n最紧张时段: {tightest['time']:.1f}h")
            print(f" 需求: {tightest['demand']:.0f} 用户, 供给: {tightest['supply']:.0f} 车位")
            print(f" 压力比: {tightest['pressure']:.2f}")

            loosest = supply_demand_valid.loc[supply_demand_valid["pressure"].idxmin()]
            print(f"\n最宽松时段: {loosest['time']:.1f}h")
            print(f" 需求: {loosest['demand']:.0f} 用户, 供给: {loosest['supply']:.0f} 车位")
            print(f" 压力比: {loosest['pressure']:.2f}")

            # 总供需统计
            avg_pressure = supply_demand_valid["pressure"].mean()
            peak_pressure = supply_demand_valid["pressure"].max()
            print(f"\n平均压力比: {avg_pressure:.2f}")
            print(f"峰值压力比: {peak_pressure:.2f}")

    return supply_demand

def analyze_walking_distance(dist_matrix, eligible, users, verbose=True):
    """分析步行距离分布"""
    # 所有可行配对的步行距离
    feasible_distances = []
    for j in range(len(users)):
        for i in range(dist_matrix.shape[1]):
            if eligible[j, i]:
                feasible_distances.append(dist_matrix[j, i])

    feasible_distances = np.array(feasible_distances)
    max_walk_values = users["max_walk"].values

```

```

if verbose:
    print("\n===== 步行距离分析 =====")
    print(f"可行配对总数: {len(feasible_distances)}")
    print(f"可行步行距离:")
    print(f"  最小: {feasible_distances.min():.2f} 米")
    print(f"  最大: {feasible_distances.max():.2f} 米")
    print(f"  平均: {feasible_distances.mean():.2f} 米")
    print(f"  中位数: {np.median(feasible_distances):.2f} 米")
    print(f"  标准差: {feasible_distances.std():.2f} 米")

    # 用户最大步行距离分布
    print(f"\n用户最大步行距离:")
    print(f"  最小: {max_walk_values.min():.2f} 米")
    print(f"  最大: {max_walk_values.max():.2f} 米")
    print(f"  平均: {max_walk_values.mean():.2f} 米")
    print(f"  中位数: {np.median(max_walk_values):.2f} 米")

    # 距离区间分布
    bins = [0, 5, 10, 15, 20, 25, 30, 50]
    labels = [f"{bins[i]}-{bins[i+1]}" for i in range(len(bins)-1)]
    dist_bins = np.digitize(feasible_distances, bins) - 1
    for b_idx in range(len(bins)-1):
        count = np.sum(dist_bins == b_idx)
        pct = 100 * count / len(feasible_distances)
        print(f"  {labels[b_idx]}米: {count} ({pct:.1f}%)")

return feasible_distances

```

```

def analyze_user_max_walk_distribution(users, verbose=True):
    """分析用户最大步行距离设置分布"""
    max_walk = users["max_walk"].values

    # 每5米一个区间
    bins = np.arange(0, max_walk.max() + 5, 5)
    hist, _ = np.histogram(max_walk, bins=bins)

    if verbose:
        print("\n===== 用户最大步行容忍度分布 =====")
        for i in range(len(hist)):
            if hist[i] > 0:
                print(f"  [{bins[i]:.0f}-{bins[i+1]:.0f}]米: {hist[i]} 用户  

                ({100*hist[i]/len(max_walk):.1f}%)")

    return bins, hist

```

```

def generate_plots(spots, slots, users, q1_result=None, q2_result=None,
q3_result=None):
    """生成可视化图表"""
    fig, axes = plt.subplots(2, 3, figsize=(18, 12))

    # 1. 车位和用户空间分布
    ax1 = axes[0, 0]
    ax1.scatter(spots["x"], spots["y"], c=pick_color(0.25), s=20, alpha=0.5,
label="车位")

```

```

    ax1.scatter(users["dest_x"], users["dest_y"], c=pick_color(0.75), s=10,
alpha=0.25, label="用户目的地")
    ax1.set_xlabel("X 坐标 (米)")
    ax1.set_ylabel("Y 坐标 (米)")
    ax1.set_title("车位与用户目的地空间分布")
    ax1.legend()
    ax1.set_aspect("equal")

# 2. 时段供需
ax2 = axes[0, 1]
sd = analyze_supply_demand(spots, slots, users, verbose=False)
ax2.plot(sd["time"], sd["supply"], "-", color=pick_color(0.25), marker="o",
linewidth=2, label="供给 (可用车位)")
ax2.plot(sd["time"], sd["demand"], "-", color=pick_color(0.75), marker="o",
linewidth=2, label="需求 (活跃用户)")
ax2.fill_between(sd["time"], sd["supply"], sd["demand"],
                 where=sd["demand"] > sd["supply"],
                 color=pick_color(0.75), alpha=0.18, label="供给缺口")
ax2.set_xlabel("时间 (小时)")
ax2.set_ylabel("数量")
ax2.set_title("时段供需分析")
ax2.legend()
ax2.grid(True, alpha=0.3)

# 3. 步行距离分布
ax3 = axes[0, 2]
dist_matrix = compute_distance_matrix(spots, users)
eligible = build_eligibility(dist_matrix, users)
feasible_distances = analyze_walking_distance(dist_matrix, eligible, users,
verbose=False)
ax3.hist(feasible_distances, bins=30, alpha=0.75, color=pick_color(0.45),
edgecolor="black")
ax3.axvline(feasible_distances.mean(), color=pick_color(0.75), linestyle="--",
            label=f"均值={feasible_distances.mean():.1f}")
ax3.set_xlabel("步行距离 (米)")
ax3.set_ylabel("频数")
ax3.set_title("可行配对步行距离分布")
ax3.legend()
ax3.grid(True, alpha=0.3)

# 4. 用户最大步行距离
ax4 = axes[1, 0]
max_walk = users["max_walk"].values
ax4.hist(max_walk, bins=20, alpha=0.75, color=pick_color(0.15),
edgecolor="black")
ax4.axvline(max_walk.mean(), color=pick_color(0.75), linestyle="--",
            label=f"均值={max_walk.mean():.1f}")
ax4.set_xlabel("最大步行距离 (米)")
ax4.set_ylabel("用户数")
ax4.set_title("用户最大步行容忍度分布")
ax4.legend()
ax4.grid(True, alpha=0.3)

# 5. 各问题结果对比
ax5 = axes[1, 1]
labels = []
n_vals = []

```

```

dist_vals = []
if q1_result and ("n_assigned" in q1_result) and ("avg_distance" in q1_result):
    labels.append("问题1\n无时间约束")
    n_vals.append(q1_result["n_assigned"])
    dist_vals.append(q1_result["avg_distance"])
if q2_result and ("n_assigned" in q2_result) and ("avg_distance" in q2_result):
    labels.append("问题2\n时间约束")
    n_vals.append(q2_result["n_assigned"])
    dist_vals.append(q2_result["avg_distance"])
if q3_result and ("n_assigned" in q3_result) and ("avg_distance" in q3_result):
    strategy = q3_result.get("strategy", "")
    if strategy:
        labels.append(f"问题3\n在线{strategy}")
    else:
        labels.append("问题3\n在线调度")
    n_vals.append(q3_result["n_assigned"])
    dist_vals.append(q3_result["avg_distance"])

if len(labels) == 0:
    print("[WARN] 各问题结果对比为空: 检查 q1_result/q2_result/q3_result 是否包含
n_assigned 和 avg_distance")
    print("  q1_result keys:", sorted(list(q1_result.keys()))) if q1_result else
None)
    print("  q2_result keys:", sorted(list(q2_result.keys()))) if q2_result else
None)
    print("  q3_result keys:", sorted(list(q3_result.keys()))) if q3_result else
None)

x = np.arange(len(labels))
width = 0.35
bars1 = ax5.bar(x - width/2, n_vals, width, label="服务用户数",
color=pick_color(0.25))
ax5_twin = ax5.twinx()
bars2 = ax5_twin.bar(x + width/2, dist_vals, width, label="平均步行距离",
color=pick_color(0.65))
ax5.set_xticks(x)
ax5.set_xticklabels(labels)
ax5.set_ylabel("服务用户数")
ax5_twin.set_ylabel("平均步行距离 (米)")
ax5.set_title("各问题结果对比")
lines1, labels1 = ax5.get_legend_handles_labels()
lines2, labels2 = ax5_twin.get_legend_handles_labels()
ax5.legend(lines1 + lines2, labels1 + labels2, loc="upper right")
ax5.grid(True, alpha=0.3)

# 6. 压力比时间序列
ax6 = axes[1, 2]
sd_valid = sd[sd["demand"] > 0].copy()
ax6.plot(sd_valid["time"], sd_valid["pressure"], "-", color=pick_color(0.85),
linewidth=2, marker="o")
ax6.axhline(1.0, color=pick_color(0.05), linestyle="--", alpha=0.6, label="供需
平衡线")
ax6.fill_between(sd_valid["time"], sd_valid["pressure"], 1,
                 where=sd_valid["pressure"] > 1,
                 color=pick_color(0.75), alpha=0.18, label="供不应求")
ax6.fill_between(sd_valid["time"], sd_valid["pressure"], 1,
                 where=sd_valid["pressure"] < 1,

```

```

        color=pick_color(0.25), alpha=0.18, label="供过于求")
ax6.set_xlabel("时间 (小时)")
ax6.set_ylabel("压力比 (需求/供给)")
ax6.set_title("时段压力比")
ax6.legend()
ax6.grid(True, alpha=0.3)
ax6.set_ylim(0, max(sd_valid["pressure"].max() * 1.2, 2))

plt.tight_layout()
plt.savefig("q4_analysis.png", dpi=150)
print("\n图表已保存: q4_analysis.png")
plt.close()

```

```

def generate_recommendations(spots, slots, users, supply_demand, verbose=True):
    """生成运营建议"""
    sd = supply_demand

    # 平均压力比
    avg_pressure = sd[sd["demand"] > 0]["pressure"].mean()
    peak_time = sd.loc[sd["pressure"].idxmax()]["time"]

    # 统计槽位特征
    slots_per_spot = slots.groupby("spot_id").size()
    multi_slot_count = (slots_per_spot > 1).sum()

    if verbose:
        print("\n" + "=" * 60)
        print("问题4: 供需匹配分析与改进建议")
        print("=" * 60)

        print(f"\n【供需匹配分析】")
        print(f"总车位: {len(spots)}")
        print(f"总用户请求: {len(users)}")
        print(f"平均时段压力比: {avg_pressure:.2f}")
        print(f"最紧张时段: {peak_time:.1f}h")
        print(f"拥有多时段的车位数: {multi_slot_count} (可错峰共享)")

        print(f"\n【改进建议】")

        print(f"\n建议1: 动态定价策略")
        print(f" 问题: 时段供需严重不均, {peak_time:.1f}h 压力比达峰值。")
        print(f" 方案:")
        print(f"    - 高峰时段({peak_time-1:.0f}-{peak_time+1:.0f}h)提高停车费率,")
        print(f"      引导非必需用户错峰出行")
        print(f"    - 低峰时段提供折扣费率, 提高车位利用率")
        print(f"    - 对预约用户提供价格锁定, 鼓励提前规划")

        print(f"\n建议2: 鼓励错峰共享")
        print(f" 问题: 拥有多时段的车位仅 {multi_slot_count} 个, 大量车位仅单一时段可用。")
        print(f" 方案:")
        print(f"    - 鼓励车位主开放更多空闲时段 (如夜间、周末)")
        print(f"    - 对开放多时段的车位主提供补贴或税收优惠")
        print(f"    - 推广\"上班族-夜班族\"车位时段互换匹配")

        print(f"\n建议3: 优化用户步行体验")

```

```

print(f"  方案: ")
print(f"      - 在用户高密度区域（基于目的地聚类）增加共享车位供给")
print(f"      - 提供精准步行导航，规划最优步行路径")
print(f"      - 对步行距离较远的用户提供积分补偿")

print(f"\n建议4: 智能预约与推荐系统")
print(f"  方案: ")
print(f"      - 基于历史数据预测未来时段的供需情况")
print(f"      - 用户提交请求时，系统推荐最可能成功的车位")
print(f"      - 若无法满足精确时间，推荐相近可用时段的车位")

return {
    "avg_pressure": avg_pressure,
    "peak_time": peak_time,
    "multi_slot_count": multi_slot_count
}

def solve_q4(q1_result=None, q2_result=None, q3_result=None, verbose=True):
    """运行问题4的全部分析"""
    if q1_result is None:
        q1_result = solve_q1(verbose=False)

    if q2_result is None:
        q2_result = solve_q2(verbose=False)

    if q3_result is None:
        q3_nearest = solve_q3(strategy="nearest", verbose=False)
        q3_balance = solve_q3(strategy="balance", verbose=False)
        q3_result = q3_nearest if q3_nearest.get("n_assigned", 0) >=
q3_balance.get("n_assigned", 0) else q3_balance

    spots, slots, users = load_data()

    supply_demand = analyze_supply_demand(spots, slots, users, verbose=verbose)

    dist_matrix = compute_distance_matrix(spots, users)
    eligible = build_eligibility(dist_matrix, users)

    # 三层供需压力指标（论文 Q4 新增:  $P(t)$  下界 /  $P_{eff}(t)$  有效压力）
    w_median = float(np.median(users["max_walk"].values))
    effective_pressure = analyze_effective_pressure(
        spots, slots, users, dist_matrix, w_median=w_median, verbose=verbose
    )

    analyze_walking_distance(dist_matrix, eligible, users, verbose=verbose)
    analyze_user_max_walk_distribution(users, verbose=verbose)

    generate_plots(spots, slots, users, q1_result, q2_result, q3_result)

    recs = generate_recommendations(spots, slots, users, supply_demand,
    verbose=verbose)

    return {
        "supply_demand": supply_demand,
        "effective_pressure": effective_pressure,
        "recommendations": recs,
    }

```

```
}
```

```
if __name__ == "__main__":  
    solve_q4(verbose=True)
```

敏感度分析 sensitivity/

sensitivity_analysis.py — 敏感度分析

```
"""敏感度分析
```

分析以下参数对结果的影响:

1. 步行距离阈值 (max_walk) - 用户可接受的最大步行距离
2. 用户停车时长 - 用户停车时间长度的影响
3. 车位时段数量 - 车位开放更多时段的影响

```
"""
```

```
import numpy as np  
import pandas as pd  
import matplotlib  
matplotlib.use("Agg")  
import matplotlib.pyplot as plt  
import sys  
import time
```

```
from utils import load_data, compute_distance_matrix, build_eligibility,  
build_time_eligibility  
from q2_time import build_spot_time_conflicts  
import pulp
```

```
from plot_theme import apply_common_style, pick_color
```

```
apply_common_style()
```

```
if hasattr(sys.stdout, "reconfigure"):  
    sys.stdout.reconfigure(encoding="utf-8")
```

```
def solve_q1_with_data(spots, slots, users, dist_matrix, eligible, verbose=False):  
    """使用给定数据求解 Q1 (避免重新加载)"""  
    n_users, n_spots = eligible.shape
```

```
    edges = []  
    for j in range(n_users):  
        for i in range(n_spots):  
            if eligible[j, i]:  
                edges.append((j, i))
```

```

    if not edges:
        return {"n_assigned": 0, "avg_distance": 0, "utilization": 0,
"total_distance": 0}

    # 阶段1: 最大基数匹配
    prob1 = pulp.LpProblem("Q1_S", pulp.LpMaximize)
    x_vars = {(j, i): pulp.LpVariable(f"x_{j}_{i}", cat="Binary") for (j, i) in
edges}
    prob1 += pulp.lpSum(x_vars.values())

    for j in range(n_users):
        incident = [x_vars[(j, i)] for (uj, i) in edges if uj == j]
        if incident:
            prob1 += pulp.lpSum(incident) <= 1

    for i in range(n_spots):
        incident = [x_vars[(j, i)] for (j, si) in edges if si == i]
        if incident:
            prob1 += pulp.lpSum(incident) <= 1

    prob1.solve(pulp.PULP_CBC_CMD(msg=False))
    if prob1.status != pulp.LpStatusOptimal:
        return {"n_assigned": 0, "avg_distance": 0, "utilization": 0,
"total_distance": 0}

    max_assigned = int(pulp.value(prob1.objective))

    # 阶段2: 最小化距离
    prob2 = pulp.LpProblem("Q1_D", pulp.LpMinimize)
    x2 = {(j, i): pulp.LpVariable(f"y_{j}_{i}", cat="Binary") for (j, i) in edges}
    prob2 += pulp.lpSum([dist_matrix[j, i] * x2[(j, i)] for (j, i) in edges])

    for j in range(n_users):
        incident = [x2[(j, i)] for (uj, i) in edges if uj == j]
        if incident:
            prob2 += pulp.lpSum(incident) <= 1

    for i in range(n_spots):
        incident = [x2[(j, i)] for (j, si) in edges if si == i]
        if incident:
            prob2 += pulp.lpSum(incident) <= 1

    prob2 += pulp.lpSum(x2.values()) == max_assigned

    prob2.solve(pulp.PULP_CBC_CMD(msg=False))
    if prob2.status != pulp.LpStatusOptimal:
        return {"n_assigned": 0, "avg_distance": 0, "utilization": 0,
"total_distance": 0}

    total = sum(dist_matrix[j, i] * pulp.value(x2[(j, i)])) for (j, i) in edges)
    avg = total / max_assigned if max_assigned > 0 else 0

    return {
        "n_assigned": max_assigned,
        "total_distance": total,
        "avg_distance": avg,
        "utilization": max_assigned / n_spots,

```

```
}
```

```
def solve_q2_with_data(spots, slots, users, dist_matrix, eligible, time_eligible,
verbose=False):
    """使用给定数据求解 Q2"""
    n_users, n_spots = time_eligible.shape

    edges = [(j, i) for j in range(n_users) for i in range(n_spots) if
time_eligible[j, i]]

    if not edges:
        return {"n_assigned": 0, "avg_distance": 0, "utilization": 0,
"total_distance": 0}

    conflict_map = build_spot_time_conflicts(users, slots, time_eligible)

    # 阶段1
    prob1 = pulp.LpProblem("Q2_S", pulp.LpMaximize)
    x = {(j, i): pulp.LpVariable(f"x_{j}_{i}", cat="Binary") for (j, i) in edges}
    prob1 += pulp.lpSum(x.values())

    for j in range(n_users):
        inc = [x[(j, i)] for (uj, i) in edges if uj == j]
        if inc:
            prob1 += pulp.lpSum(inc) <= 1

    for i in range(n_spots):
        for (j1, j2) in conflict_map[i]:
            if (j1, i) in x and (j2, i) in x:
                prob1 += x[(j1, i)] + x[(j2, i)] <= 1

    prob1.solve(pulp.PULP_CBC_CMD(msg=False))
    if prob1.status != pulp.LpStatusOptimal:
        return {"n_assigned": 0, "avg_distance": 0, "utilization": 0,
"total_distance": 0}

    max_assigned = int(pulp.value(prob1.objective))

    # 阶段2
    prob2 = pulp.LpProblem("Q2_D", pulp.LpMinimize)
    x2 = {(j, i): pulp.LpVariable(f"y_{j}_{i}", cat="Binary") for (j, i) in edges}
    prob2 += pulp.lpSum([dist_matrix[j, i] * x2[(j, i)] for (j, i) in edges])

    for j in range(n_users):
        inc = [x2[(j, i)] for (uj, i) in edges if uj == j]
        if inc:
            prob2 += pulp.lpSum(inc) <= 1

    for i in range(n_spots):
        for (j1, j2) in conflict_map[i]:
            if (j1, i) in x2 and (j2, i) in x2:
                prob2 += x2[(j1, i)] + x2[(j2, i)] <= 1

    prob2 += pulp.lpSum(x2.values()) == max_assigned
    prob2.solve(pulp.PULP_CBC_CMD(msg=False))
    if prob2.status != pulp.LpStatusOptimal:
```

```

        return {"n_assigned": 0, "avg_distance": 0, "utilization": 0,
"total_distance": 0}

    total = sum(dist_matrix[j, i] * pulp.value(x2[(j, i)])) for (j, i) in edges)
    avg = total / max_assigned if max_assigned > 0 else 0

    return {
        "n_assigned": max_assigned,
        "total_distance": total,
        "avg_distance": avg,
        "utilization": max_assigned / n_spots,
    }

def sensitivity_walk_distance():
    """敏感度分析1: 步行距离阈值"""
    print("\n" + "=" * 70)
    print("敏感度分析 1: 步行距离阈值 (max_walk) 倍数变化")
    print("=" * 70)

    spots, slots, users = load_data()
    dist_matrix = compute_distance_matrix(spots, users)
    original_max_walk = users["max_walk"].copy()

    multipliers = [0.5, 0.75, 1.0, 1.25, 1.5, 2.0]
    results = []

    for mult in multipliers:
        users["max_walk"] = original_max_walk * mult
        eligible = build_eligibility(dist_matrix, users)

        print(f"\n>>> 步行距离倍数 = {mult:.2f}, 平均阈值 =
{users['max_walk'].mean():.2f}m")
        print(f"    可行边数: {eligible.sum()}")

        # Q1
        t0 = time.time()
        r1 = solve_q1_with_data(spots, slots, users, dist_matrix, eligible)
        t1 = time.time() - t0

        # Q2 (需要重新计算 time_eligible)
        time_eligible = build_time_eligibility(users, slots, eligible)
        t0 = time.time()
        r2 = solve_q2_with_data(spots, slots, users, dist_matrix, eligible,
time_eligible)
        t2 = time.time() - t0

        print(f"    Q1: 分配={r1['n_assigned']}, 平均距离={r1['avg_distance']:.2f}m,
耗时={t1:.2f}s")
        print(f"    Q2: 分配={r2['n_assigned']}, 平均距离={r2['avg_distance']:.2f}m,
耗时={t2:.2f}s")

        results.append({
            "multiplier": mult,
            "q1_assigned": r1["n_assigned"],
            "q1_avg_dist": r1["avg_distance"],
            "q1_util": r1["utilization"] * 100,

```

```

        "q2_assigned": r2["n_assigned"],
        "q2_avg_dist": r2["avg_distance"],
        "q2_util": r2["utilization"] * 100,
        "q1_time": t1,
        "q2_time": t2,
    })

users["max_walk"] = original_max_walk

# 打印汇总
print("\n" + "=" * 70)
print("步行距离敏感度结果汇总")
print("=" * 70)
print(f"{'倍数':<8} {'Q1分配':<8} {'Q1距离':<10} {'Q2分配':<8} {'Q2距离':<10} {'Q2/Q1比':<10}")
print("-" * 70)
for r in results:
    ratio = r["q2_assigned"] / r["q1_assigned"] if r["q1_assigned"] > 0 else 0
    print(f"{r['multiplier']:<8.2f} {r['q1_assigned']:<8} {r['q1_avg_dist']:<10.2f} "
          f"{r['q2_assigned']:<8} {r['q2_avg_dist']:<10.2f} {ratio:<10.2%}")

return pd.DataFrame(results)

def sensitivity_user_duration():
    """敏感度分析2: 用户停车时长变化"""
    print("\n" + "=" * 70)
    print("敏感度分析 2: 用户停车时长变化")
    print("=" * 70)

    spots, slots, users = load_data()
    dist_matrix = compute_distance_matrix(spots, users)
    eligible = build_eligibility(dist_matrix, users)

    original_arrival = users["arrival"].copy()
    original_departure = users["departure"].copy()

    duration_factors = [0.5, 0.75, 1.0, 1.25, 1.5]
    results = []

    for factor in duration_factors:
        # 调整离开时间（保持开始时间不变）
        duration = original_departure - original_arrival
        users["departure"] = original_arrival + duration * factor

        time_eligible = build_time_eligibility(users, slots, eligible)
        n_compatible = time_eligible.sum()

        print(f"\n>>> 时长倍数 = {factor:.2f}, 平均时长 = {(users['departure'] - users['arrival']).mean():.2f}h")
        print(f"    时间兼容边数: {n_compatible}")

        t0 = time.time()
        r2 = solve_q2_with_data(spots, slots, users, dist_matrix, eligible, time_eligible)
        t2 = time.time() - t0

```

```

        print(f"      Q2: 分配={r2['n_assigned']}, 平均距离={r2['avg_distance']:.2f}m,
耗时={t2:.2f}s")

    results.append({
        "duration_factor": factor,
        "avg_duration": (users["departure"] - users["arrival"]).mean(),
        "q2_assigned": r2["n_assigned"],
        "q2_avg_dist": r2["avg_distance"],
        "q2_util": r2["utilization"] * 100,
        "compatible_edges": int(n_compatible),
        "time": t2,
    })

users["arrival"] = original_arrival
users["departure"] = original_departure

# 打印汇总
print("\n" + "=" * 70)
print("用户停车时长敏感度结果")
print("=" * 70)
print(f"{'时长倍数':<10} {'平均时长(h)':<12} {'Q2分配':<8} {'Q2距离':<10} {'兼容'
边':<10}")
print("-" * 70)
for r in results:
    print(f"{r['duration_factor']:<10.2f} {r['avg_duration']:<12.2f} "
          f"{r['q2_assigned']:<8} {r['q2_avg_dist']:<10.2f} "
          f"{r['compatible_edges']:<10}")

return pd.DataFrame(results)

def sensitivity_slot_extension():
    """敏感度分析3: 车位时段扩展"""
    print("\n" + "=" * 70)
    print("敏感度分析 3: 车位可用时段扩展（错时共享）")
    print("=" * 70)

    spots, slots, users = load_data()
    dist_matrix = compute_distance_matrix(spots, users)
    eligible = build_eligibility(dist_matrix, users)

    # 原始结果
    print("\n>>> 原始时段:")
    time_eligible_orig = build_time_eligibility(users, slots, eligible)
    print(f"      时间兼容边数: {time_eligible_orig.sum()}")
    t0 = time.time()
    r2_orig = solve_q2_with_data(spots, slots, users, dist_matrix, eligible,
time_eligible_orig)
    t_orig = time.time() - t0
    print(f"      Q2: 分配={r2_orig['n_assigned']}, 平均距离=
{r2_orig['avg_distance']:.2f}m, 耗时={t_orig:.2f}s")

    # 全天开放
    extended_slots_full = pd.DataFrame([{"spot_id": sid, "start": 8.0, "end": 22.0}
                                         for sid in spots["spot_id"]])
    print("\n>>> 全天开放时段 (8-22) - 上限场景:")

```

```

time_eligible_full = build_time_eligibility(users, extended_slots_full,
eligible)
print(f"    时间兼容边数: {time_eligible_full.sum()}")
t0 = time.time()
r2_full = solve_q2_with_data(spots, extended_slots_full, users, dist_matrix,
eligible, time_eligible_full)
t_full = time.time() - t0
print(f"    Q2: 分配={r2_full['n_assigned']}, 平均距离=
{r2_full['avg_distance']:.2f}m, 耗时={t_full:.2f}s")

# 单时段补充反向时段
print("\n>>> 单时段车位补充反向时段 (中等场景):")
spot_slot_count = slots.groupby("spot_id").size()
single_slot_spots = spot_slot_count[spot_slot_count == 1].index

new_rows = []
for spot_id in single_slot_spots:
    original_slot = slots[slots["spot_id"] == spot_id].iloc[0]
    s, e = original_slot["start"], original_slot["end"]
    if e <= 13:
        new_rows.append({"spot_id": spot_id, "start": 14.0, "end": 18.0})
    elif s >= 13:
        new_rows.append({"spot_id": spot_id, "start": 8.0, "end": 12.0})
    else:
        new_rows.append({"spot_id": spot_id, "start": 18.0, "end": 22.0})

extended_slots_mid = pd.concat([slots, pd.DataFrame(new_rows)],
ignore_index=True)

print(f"    新增时段数: {len(new_rows)}")
time_eligible_mid = build_time_eligibility(users, extended_slots_mid, eligible)
print(f"    时间兼容边数: {time_eligible_mid.sum()}")
t0 = time.time()
r2_mid = solve_q2_with_data(spots, extended_slots_mid, users, dist_matrix,
eligible, time_eligible_mid)
t_mid = time.time() - t0
print(f"    Q2: 分配={r2_mid['n_assigned']}, 平均距离=
{r2_mid['avg_distance']:.2f}m, 耗时={t_mid:.2f}s")

# 汇总
print("\n" + "=" * 70)
print("时段扩展场景对比")
print("=" * 70)
print(f"{'场景':<25} {'分配数':<10} {'平均距离':<12} {'利用率':<10}")
print("-" * 70)
print(f"{'原始时段':<25} {r2_orig['n_assigned']:<10} {r2_orig['avg_distance']:
<12.2f} {r2_orig['utilization']*100:.2f}%")
print(f"{'单时段补充反向':<25} {r2_mid['n_assigned']:<10}
{r2_mid['avg_distance']:<12.2f} {r2_mid['utilization']*100:.2f}%")
print(f"{'全天 8-22 开放':<25} {r2_full['n_assigned']:<10}
{r2_full['avg_distance']:<12.2f} {r2_full['utilization']*100:.2f}%")

if r2_orig["n_assigned"] > 0:
    improvement_mid = (r2_mid["n_assigned"] - r2_orig["n_assigned"]) /
r2_orig["n_assigned"] * 100
    improvement_full = (r2_full["n_assigned"] - r2_orig["n_assigned"]) /
r2_orig["n_assigned"] * 100

```

```
print(f"\n中等扩展提升: {improvement_mid:+.2f}%")
print(f"全天开放提升: {improvement_full:+.2f}%")
```

```
return {
    "original": r2_orig,
    "mid_extension": r2_mid,
    "full_extension": r2_full,
}
```

```
def plot_results(walk_df, dur_df, slot_results):
    """生成敏感度分析图表"""
    fig, axes = plt.subplots(2, 2, figsize=(15, 11))

    # 1. 步行距离敏感度 - 分配数
    ax = axes[0, 0]
    ax.plot(walk_df["multiplier"], walk_df["q1_assigned"], 'o-', label='Q1(静态)',
            linewidth=2, markersize=8, color=pick_color(0.25))
    ax.plot(walk_df["multiplier"], walk_df["q2_assigned"], 's-', label='Q2(时间约束)',
            linewidth=2, markersize=8, color=pick_color(0.75))
    ax.axvline(x=1.0, color=pick_color(0.05), linestyle='--', alpha=0.5)
    ax.set_xlabel('步行距离阈值倍数', fontsize=12)
    ax.set_ylabel('分配用户数', fontsize=12)
    ax.set_title('步行距离阈值对分配数的影响', fontsize=13, fontweight='bold')
    ax.legend(fontsize=10)
    ax.grid(True, alpha=0.3)

    # 2. 步行距离对平均距离的影响
    ax = axes[0, 1]
    ax.plot(walk_df["multiplier"], walk_df["q1_avg_dist"], 'o-', label='Q1',
            linewidth=2, markersize=8, color=pick_color(0.25))
    ax.plot(walk_df["multiplier"], walk_df["q2_avg_dist"], 's-', label='Q2',
            linewidth=2, markersize=8, color=pick_color(0.75))
    ax.axvline(x=1.0, color=pick_color(0.05), linestyle='--', alpha=0.5)
    ax.set_xlabel('步行距离阈值倍数', fontsize=12)
    ax.set_ylabel('平均步行距离 (米)', fontsize=12)
    ax.set_title('步行距离阈值对平均距离的影响', fontsize=13, fontweight='bold')
    ax.legend(fontsize=10)
    ax.grid(True, alpha=0.3)

    # 3. 用户停车时长敏感度
    ax = axes[1, 0]
    ax.plot(dur_df["duration_factor"], dur_df["q2_assigned"], 'o-',
            color=pick_color(0.55), linewidth=2, markersize=8, label='Q2分配数')
    ax.axvline(x=1.0, color=pick_color(0.05), linestyle='--', alpha=0.5)
    ax.set_xlabel('用户停车时长倍数', fontsize=12)
    ax.set_ylabel('Q2 分配用户数', fontsize=12, color=pick_color(0.55))
    ax.tick_params(axis='y', labelcolor=pick_color(0.55))
    ax.set_title('用户停车时长对分配数的影响', fontsize=13, fontweight='bold')
    ax.grid(True, alpha=0.3)

    ax2 = ax.twinx()
    ax2.plot(dur_df["duration_factor"], dur_df["compatible_edges"], 's--',
            color=pick_color(0.85), alpha=0.7, label='兼容边数')
    ax2.set_ylabel('时间兼容边数', fontsize=12, color=pick_color(0.85))
    ax2.tick_params(axis='y', labelcolor=pick_color(0.85))
```

4. 时段扩展场景对比

```
ax = axes[1, 1]
scenarios = ['原始时段', '单时段\n补充反向', '全天\n8-22开放']
counts = [
    slot_results["original"]["n_assigned"],
    slot_results["mid_extension"]["n_assigned"],
    slot_results["full_extension"]["n_assigned"],
]
colors = [pick_color(0.25), pick_color(0.55), pick_color(0.85)]
bars = ax.bar(scenarios, counts, color=colors, alpha=0.8, edgecolor='black')
ax.set_ylabel('分配用户数', fontsize=12)
ax.set_title('车位时段扩展场景对比', fontsize=13, fontweight='bold')
ax.grid(True, alpha=0.3, axis='y')
for bar, count in zip(bars, counts):
    ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 3,
            f'{count}', ha='center', fontsize=11, fontweight='bold')

plt.tight_layout()
plt.savefig("sensitivity_analysis.png", dpi=200, bbox_inches='tight')
print("\n敏感度分析图已保存: sensitivity_analysis.png")
```

```
def main():
    print("\n" + "=" * 70)
    print("B 题 - 敏感度分析")
    print("=" * 70)

    t_start = time.time()

    walk_df = sensitivity_walk_distance()
    dur_df = sensitivity_user_duration()
    slot_results = sensitivity_slot_extension()

    plot_results(walk_df, dur_df, slot_results)

    walk_df.to_csv("sensitivity_walk_distance.csv", index=False, encoding="utf-8-
sig")
    dur_df.to_csv("sensitivity_user_duration.csv", index=False, encoding="utf-8-
sig")

    # — 假设鲁棒性检验（论文新增：时间噪声 + 取消率扰动）
    print("\n" + "=" * 70)
    print("假设鲁棒性检验（调用 robustness/robustness_test.py）")
    print("=" * 70)
    import os, subprocess, sys
    robustness_script = os.path.normpath(os.path.join(
        os.path.dirname(os.path.abspath(__file__)), "..", "robustness",
"robustness_test.py"
    ))
    if os.path.exists(robustness_script):
        result = subprocess.run(
            [sys.executable, robustness_script],
            capture_output=True, text=True, encoding="utf-8", errors="replace"
        )
        print(result.stdout)
        if result.returncode != 0:
            print("[WARN] robustness_test.py 返回非零退出码: ")
```

```

        print(result.stderr[:500])
    else:
        print(f"[SKIP] 未找到鲁棒性检验脚本: {robustness_script}")

    print("\n" + "=" * 70)
    print("敏感度分析完成")
    print("=" * 70)
    print(f"\n总耗时: {time.time() - t_start:.2f} 秒")
    print("\n输出文件:")
    print("  - sensitivity_analysis.png           - 综合分析图")
    print("  - sensitivity_walk_distance.csv       - 步行距离敏感度数据")
    print("  - sensitivity_user_duration.csv       - 用户时长敏感度数据")
    print("  - ../robustness/robustness_A_time_noise.csv - 时间噪声鲁棒性结果")
    print("  - ../robustness/robustness_B_cancellation.csv - 取消率鲁棒性结果")

if __name__ == "__main__":
    main()

```

q4_analysis/q4_heatmap.py — 空间-时间压力热力图

"""空间-时间压力热力图分析

将空间划分为网格，对每个网格和时刻计算供需压力比，
直观展示哪些区域、哪些时段停车压力最大。

输出: q4_heatmap.png (多子图热力图)
"""

```

import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.colors import LogNorm
from matplotlib import cm
from matplotlib.colors import LightSource
from mpl_toolkits.mplot3d import Axes3D # noqa: F401
import sys

from utils import load_data, compute_distance_matrix
from plot_theme import CMAP, apply_common_style, pick_color

apply_common_style()

if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")

```

```

def build_spatial_temporal_pressure(spots, slots, users, grid_size=10,
radius=15.0):
    """
    构建空间-时间压力矩阵。

    参数:
        spots: 车位数据框
        slots: 时段数据框
        users: 用户数据框
        grid_size: 每维网格数 (grid_size x grid_size)
        radius: 可达半径 (米), 用于计算"可达供给"

    返回:
        (grid_x, grid_y, times, demand_grid, supply_grid, reachable_grid,
        pressure_grid)
        demand_grid[gx, gy, t_idx]: 网格(gx, gy)在时刻t的需求密度
        supply_grid[gx, gy, t_idx]: 网格(gx, gy)在时刻t的原始供给
        reachable_grid[gx, gy, t_idx]: 网格(gx, gy)在时刻t的可达供给
        pressure_grid[gx, gy, t_idx]: 需求/可达供给压力比
    """
    # 空间范围
    x_min, x_max = 0, 100
    y_min, y_max = 0, 100
    cell_w = (x_max - x_min) / grid_size # 格子宽度
    cell_h = (y_max - y_min) / grid_size

    # 网格中心坐标
    grid_x = np.linspace(x_min + cell_w / 2, x_max - cell_w / 2, grid_size)
    grid_y = np.linspace(y_min + cell_h / 2, y_max - cell_h / 2, grid_size)

    # 时间采样点 (半开区间, 每小时一个点)
    times = np.arange(8, 22, 1) # [8, 9, ..., 21]
    n_times = len(times)
    n_cells = grid_size * grid_size

    # 初始化矩阵
    demand_grid = np.zeros((grid_size, grid_size, n_times))
    supply_grid = np.zeros((grid_size, grid_size, n_times))
    reachable_grid = np.zeros((grid_size, grid_size, n_times))
    pressure_grid = np.full((grid_size, grid_size, n_times), np.nan)

    # ---- 用户需求映射 ----
    # 每个用户的目的地坐标映射到网格
    user_positions = users[["dest_x", "dest_y"]].values # (500, 2)
    user_arrivals = users["arrival"].values
    user_departures = users["departure"].values
    user_max_walk = users["max_walk"].values

    # 为每个用户分配网格
    for j in range(len(users)):
        ux, uy = user_positions[j]
        gx = int(np.clip((ux - x_min) / cell_w, 0, grid_size - 1))
        gy = int(np.clip((uy - y_min) / cell_h, 0, grid_size - 1))

        for t_idx, t in enumerate(times):
            if user_arrivals[j] <= t < user_departures[j]:

```

```

        demand_grid[gx, gy, t_idx] += 1

# ---- 车位供给映射 ----
spot_positions = spots[["x", "y"]].values # (300, 2)

# 构建 slot lookup: spot_id -> [(start, end), ...]
slot_map = {}
for _, row in slots.iterrows():
    sid = row["spot_id"]
    if sid not in slot_map:
        slot_map[sid] = []
    slot_map[sid].append((row["start"], row["end"]))

# 计算每个车位的网格位置，以及每个网格到所有车位的距离
spot_grid_indices = []
for sid_idx, sid in enumerate(spots["spot_id"]):
    sx, sy = spot_positions[sid_idx]
    gx = int(np.clip((sx - x_min) / cell_w, 0, grid_size - 1))
    gy = int(np.clip((sy - y_min) / cell_h, 0, grid_size - 1))
    spot_grid_indices.append((gx, gy, sx, sy))

# 计算所有网格中心到所有车位的距离矩阵 (n_cells x n_spots)
cell_centers = np.array([[gx, gy] for gy in grid_y for gx in grid_x]) # (100,
2)
cell_centers_swapped = cell_centers[:, [1, 0]] # 修正: 需要 [x, y]
# 重新构建正确的网格中心
cell_positions = []
for gy in grid_y:
    for gx in grid_x:
        cell_positions.append([gx, gy])
cell_positions = np.array(cell_positions) # (100, 2) = [x, y]

# 计算距离矩阵: 每个网格中心到每个车位
spot_coords = spot_positions # (300, 2)
from scipy.spatial.distance import cdist
dist_cell_to_spot = cdist(cell_positions, spot_coords, metric="euclidean") #
(100, 300)

# 对于每个网格、每个时间、每个车位，检查是否可达且可用
for gx_idx in range(grid_size):
    for gy_idx in range(grid_size):
        cell_idx = gx_idx + gy_idx * grid_size # flatten: column-major (gx
varies fastest)

        for t_idx, t in enumerate(times):
            # 原始供给: 车位在该时刻可用
            raw_count = 0
            reachable_count = 0

            for sid_idx, sid in enumerate(spots["spot_id"]):
                # 检查时刻 t 是否在车位可用时段内
                if sid not in slot_map:
                    continue
                is_available = False
                for s, e in slot_map[sid]:
                    if s <= t < e:
                        is_available = True

```

```

        break
    if not is_available:
        continue

    raw_count += 1

    # 检查该车位是否从网格中心可达
    d = dist_cell_to_spot[cell_idx, sid_idx]
    # 使用该网格内用户的最大步行距离中位数（若无用户则用全局中位数）
    if d <= radius:
        reachable_count += 1

    supply_grid[gx_idx, gy_idx, t_idx] = raw_count
    reachable_grid[gx_idx, gy_idx, t_idx] = reachable_count

    # 压力比
    d_val = demand_grid[gx_idx, gy_idx, t_idx]
    s_val = reachable_count
    if d_val > 0 and s_val > 0:
        pressure_grid[gx_idx, gy_idx, t_idx] = d_val / s_val
    elif d_val > 0 and s_val == 0:
        pressure_grid[gx_idx, gy_idx, t_idx] = np.inf

    return grid_x, grid_y, times, demand_grid, supply_grid, reachable_grid,
    pressure_grid

def plot_heatmap(spots, slots, users, grid_x, grid_y, times,
                 demand_grid, supply_grid, reachable_grid, pressure_grid):
    """生成空间-时间压力热力图"""
    grid_size = len(grid_x)
    n_times = len(times)

    # 选择关键时间点展示
    key_hours = [8, 10, 12, 14, 16, 18, 20]
    key_indices = [np.argmin(np.abs(times - h)) for h in key_hours]

    n_plots = len(key_hours)
    # 2行布局：第1行上午，第2行下午，7张图用2x4（空一个）
    fig, axes = plt.subplots(2, 4, figsize=(22, 12))
    axes_flat = axes.flatten()

    # 颜色映射：使用感知均匀的渐变色板，便于打印和色盲友好
    norm = plt.Normalize(vmin=0.1, vmax=4.0)
    cmap = CMAP
    cmap.set_bad('#f0f0f0') # NaN/无数据格为浅灰

    for idx, (ax, t_idx) in enumerate(zip(axes_flat[:n_plots], key_indices)):
        hour = times[t_idx]
        pressure_slice = pressure_grid[:, :, t_idx].copy()

        # 标记无供给但有需求的单元格为 NaN（灰色显示）
        no_supply_mask = ~np.isfinite(pressure_slice)
        pressure_slice[no_supply_mask] = np.nan
        # 无需求的单元格也显示灰色
        zero_demand_mask = demand_grid[:, :, t_idx] == 0
        pressure_slice[zero_demand_mask] = np.nan

```

```

# 绘制压力热力图，降低透明度让颜色更清晰/接近实心
im = ax.imshow(pressure_slice.T, origin='lower',
                extent=[grid_x[0] - (grid_x[1]-grid_x[0])/2, grid_x[-1] +
                        (grid_x[1]-grid_x[0])/2,
                        grid_y[0] - (grid_y[1]-grid_y[0])/2, grid_y[-1] +
                        (grid_y[1]-grid_y[0])/2],
                cmap=cmap, aspect='equal', norm=norm, alpha=0.95)

# 叠加散点
ax.scatter(spots["x"], spots["y"], c=pick_color(0.25), s=12, alpha=0.45,
           label='车位', marker='s', edgecolors='white', linewidth=0.3)
ax.scatter(users["dest_x"], users["dest_y"], c=pick_color(0.75), s=4,
           alpha=0.12, label='用户目的地')

# 在无供给但有需求的单元格上标注 X
no_supply_with_demand = no_supply_mask & ~zero_demand_mask
if no_supply_with_demand.any():
    n_xy = np.where(no_supply_with_demand)
    for gx_i, gy_i in zip(n_xy[0], n_xy[1]):
        cx = grid_x[gx_i]
        cy = grid_y[gy_i]
        d_val = demand_grid[gx_i, gy_i, t_idx]
        ax.text(cx, cy, f'X\n{d_val:.0f}人', ha='center', va='center',
                fontsize=6, fontweight='bold', color=pick_color(0.85),
                bbox=dict(boxstyle='round,pad=0.1',
                          facecolor='white', alpha=0.7))

# 在供需正常的单元格标注 d/s
normal_cells = ~zero_demand_mask & ~no_supply_mask
if normal_cells.any():
    n_xy = np.where(normal_cells)
    for gx_i, gy_i in zip(n_xy[0], n_xy[1]):
        cx = grid_x[gx_i]
        cy = grid_y[gy_i]
        d_val = demand_grid[gx_i, gy_i, t_idx]
        s_val = reachable_grid[gx_i, gy_i, t_idx]
        p_val = pressure_grid[gx_i, gy_i, t_idx]
        ax.text(cx, cy, f'{d_val:.0f}/{s_val:.0f}\n{p_val:.1f}x',
                ha='center', va='center',
                fontsize=6, fontweight='bold',
                color='black' if p_val < 2 else 'darkred', alpha=0.8,
                bbox=dict(boxstyle='round,pad=0.15',
                          facecolor='white', alpha=0.55))

ax.set_title(f'{hour:.0f}:00', fontsize=13, fontweight='bold')
ax.set_xlim(0, 100)
ax.set_ylim(0, 100)
ax.set_xticks(np.arange(0, 101, 20))
ax.set_yticks(np.arange(0, 101, 20))
ax.grid(True, alpha=0.2, linestyle='--', color='#B0B0B0')

# 隐藏多余的子图
for idx in range(n_plots, len(axes_flat)):
    axes_flat[idx].set_visible(False)

# 添加图例（取最后一个子图位置）

```

```

handles = [
    plt.Line2D([0], [0], marker='s', color='w',
markerfacecolor=pick_color(0.25),
                markersize=8, label='车位'),
    plt.Line2D([0], [0], marker='o', color='w',
markerfacecolor=pick_color(0.75),
                markersize=5, label='用户目的地'),
    plt.Line2D([0], [0], marker='s', color='w',
                markerfacecolor='#D9D9D9', markersize=8, label='无车位/无需
求'),
]
axes_flat[6].legend(handles=handles, loc='center', fontsize=9,
                    framealpha=0.8, title='图例', title_fontsize=10)
# 把第7个子图用作文本说明
axes_flat[7].axis('off')

# 共用颜色条
cbar_ax = fig.add_axes([0.92, 0.15, 0.012, 0.7])
cbar = fig.colorbar(im, cax=cbar_ax)
cbar.set_label('压力比 (需求/可达供给)', fontsize=11)
cbar.ax.tick_params(labelsize=9)

plt.suptitle('共享停车位空间-时间压力热力图', fontsize=16, fontweight='bold',
y=0.98)
plt.tight_layout(rect=[0, 0, 0.9, 0.95])
plt.savefig("q4_heatmap.png", dpi=200, bbox_inches='tight')
print("空间-时间热力图已保存: q4_heatmap.png")

return fig

```

```

def plot_demand_supply_summary(grid_x, grid_y, times,
                             demand_grid, supply_grid, reachable_grid,
pressure_grid):
    """生成供需汇总图：时间-空间聚合"""
    grid_size = len(grid_x)
    n_times = len(times)

    fig, axes = plt.subplots(2, 2, figsize=(14, 11))

    # 1. 总需求 vs 总供给（时间维度）
    ax = axes[0, 0]
    total_demand = demand_grid.sum(axis=(0, 1))
    total_supply = supply_grid.sum(axis=(0, 1))
    total_reachable = reachable_grid.sum(axis=(0, 1))

    ax.plot(times, total_demand, '-o', color=pick_color(0.75), label='总需求（活跃用
户）', linewidth=2, markersize=6)
    ax.plot(times, total_reachable, '-s', color=pick_color(0.55), label='总可达供给
（车位）', linewidth=2, markersize=6)
    ax.plot(times, total_supply, '--^', color=pick_color(0.25), label='总供给（全部
车位）', linewidth=2, markersize=6, alpha=0.5)
    ax.set_xlabel('时间 (h)', fontsize=12)
    ax.set_ylabel('数量', fontsize=12)
    ax.set_title('时空聚合：需求 vs 供给', fontsize=13, fontweight='bold')
    ax.legend(fontsize=10)
    ax.grid(True, alpha=0.3)

```

```

ax.set_xticks(times[::2])

# 2. 累计压力分布（空间维度）
ax = axes[0, 1]
peak_pressure = np.nanmax(np.where(np.isfinite(pressure_grid), pressure_grid,
0), axis=2)
im = ax.imshow(peak_pressure.T, origin='lower',
                extent=[grid_x[0] - 5, grid_x[-1] + 5,
                        grid_y[0] - 5, grid_y[-1] + 5],
                cmap=CMAP, aspect='equal')
ax.set_xlabel('X 坐标', fontsize=12)
ax.set_ylabel('Y 坐标', fontsize=12)
ax.set_title('各网格峰值压力比', fontsize=13, fontweight='bold')
plt.colorbar(im, ax=ax, label='峰值压力比')

# 3. 总需求空间分布
ax = axes[1, 0]
total_demand_spatial = demand_grid.sum(axis=2).T
im2 = ax.imshow(total_demand_spatial, origin='lower',
                 extent=[grid_x[0] - 5, grid_x[-1] + 5,
                         grid_y[0] - 5, grid_y[-1] + 5],
                 cmap=CMAP, aspect='equal')
ax.set_xlabel('X 坐标', fontsize=12)
ax.set_ylabel('Y 坐标', fontsize=12)
ax.set_title('累计需求密度', fontsize=13, fontweight='bold')
plt.colorbar(im2, ax=ax, label='累计活跃用户数')

# 4. 压力比箱线图
ax = axes[1, 1]
pressure_valid = []
time_labels = []
for t_idx, t in enumerate(times):
    p_slice = pressure_grid[:, :, t_idx].flatten()
    p_valid = p_slice[np.isfinite(p_slice) & (p_slice > 0)]
    if len(p_valid) > 0:
        pressure_valid.append(p_valid)
        time_labels.append(f'{t:.0f}h')

bp = ax.boxplot(pressure_valid, labels=time_labels, patch_artist=True,
showfliers=False)
for patch in bp['boxes']:
    patch.set_facecolor(pick_color(0.25))
    patch.set_alpha(0.7)

ax.axhline(y=1.0, color=pick_color(0.05), linestyle='--', alpha=0.7, label='供需平衡线')
ax.set_xlabel('时间', fontsize=12)
ax.set_ylabel('网格压力比分布', fontsize=12)
ax.set_title('各时刻网格压力分布', fontsize=13, fontweight='bold')
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("q4_heatmap_summary.png", dpi=300, bbox_inches='tight')
print("供需汇总图已保存: q4_heatmap_summary.png")

def save_svg_slice(pressure_slice, grid_x, grid_y, outpath, hour_label):

```

```

"""Save a single 2D压力切片为SVG, 以便论文嵌入。"""
fig, ax = plt.subplots(figsize=(6,5))
norm = plt.Normalize(vmin=0.1, vmax=4.0)
im = ax.imshow(pressure_slice.T, origin='lower',
               extent=[grid_x[0] - 5, grid_x[-1] + 5,
                       grid_y[0] - 5, grid_y[-1] + 5],
               cmap=CMAP, aspect='equal', norm=norm)
ax.set_title(f'Pressure Slice @ {hour_label}h', fontsize=12)
ax.axis('off')
plt.tight_layout()
plt.savefig(outputpath, format='svg', bbox_inches='tight')
plt.close(fig)
print(f"Saved SVG slice: {outputpath}")

```

```

def plot_heatmap_3d(grid_x, grid_y, times, pressure_grid, t_idx=None,
outputpath="q4_heatmap_3d.png"):
    """将单时刻的压力热力图渲染为三维表面图并保存"""
    # 选择时刻
    if t_idx is None:
        # 默认选择压力峰值时刻
        valid_means = np.nanmean(np.where(np.isfinite(pressure_grid),
pressure_grid, np.nan), axis=(0,1))
        t_idx = int(np.nanargmax(valid_means))

    X, Y = np.meshgrid(grid_x, grid_y)
    Z = pressure_grid[:, :, t_idx].T # transpose to match X,Y

    fig = plt.figure(figsize=(12, 8))
    ax = fig.add_subplot(111, projection='3d')

    # Professional pane/grid styling
    try:
        ax.xaxis.pane.set_facecolor((1.0, 1.0, 1.0, 0.0))
        ax.yaxis.pane.set_facecolor((1.0, 1.0, 1.0, 0.0))
        ax.zaxis.pane.set_facecolor((1.0, 1.0, 1.0, 0.0))
        for axis in (ax.xaxis, ax.yaxis, ax.zaxis):
            axis._axinfo["grid"]["color"] = (0.78, 0.78, 0.78, 0.35)
            axis._axinfo["grid"]["linewidth"] = 0.8
    except Exception:
        pass

    # mask invalid
    mask = ~np.isfinite(Z)
    Z_masked = np.ma.array(Z, mask=mask)

    # build full-resolution grid for smooth surface
    try:
        xi = np.linspace(grid_x.min(), grid_x.max(), 200)
        yi = np.linspace(grid_y.min(), grid_y.max(), 200)
        XI, YI = np.meshgrid(xi, yi)
        # use simple nearest-neighbor interpolation from coarse grid to fine grid
        from scipy.interpolate import griddata
        points = np.array([(x, y) for x in grid_x for y in grid_y])
        values = Z_masked.data.flatten().T
        # mask out invalid
        valid = np.isfinite(values)

```

```

        if valid.sum() >= 4:
            ZI = griddata(points[valid], values[valid], (XI, YI), method='cubic')
        else:
            ZI = griddata(points, values, (XI, YI), method='nearest')
    except Exception:
        # fallback: simple upsample by nearest
        XI, YI = np.meshgrid(grid_x, grid_y)
        ZI = Z_masked

# apply a light source shading to give a 3D terrain feel
ls = LightSource(azdeg=315, altdeg=45)
rgb = ls.shade(ZI, cmap=CMAP, vert_exag=1.0, blend_mode='soft')

# plot surface with shaded facecolors
surf = ax.plot_surface(XI, YI, ZI, rstride=1, cstride=1, facecolors=rgb,
                       linewidth=0, antialiased=True, shade=False)

# add contour projection at base for readability
zmin = np.nanmin(ZI)
offset = zmin - (np.nanmax(ZI) - zmin) * 0.25
try:
    cs = ax.contourf(XI, YI, ZI, zdir='z', offset=offset, cmap=CMAP, levels=12,
alpha=0.7)
except Exception:
    pass

ax.set_xlabel('X (m)')
ax.set_ylabel('Y (m)')
ax.set_zlabel('Pressure Ratio')
hour = times[t_idx]
ax.set_title(f'Spatio-temporal Pressure 3D Surface ({hour}:00)', fontsize=14)

# style adjustments
ax.set_box_aspect((np.ptp(grid_x), np.ptp(grid_y), max(1e-6, np.nanmax(ZI)-
offset)))
ax.view_init(elev=30, azimuth=-60)

# colorbar as separate mappable
mappable = cm.ScalarMappable(cmap=CMAP)
mappable.set_array(ZI)
fig.colorbar(mappable, shrink=0.6, aspect=12, pad=0.08, label='Pressure Ratio')

plt.tight_layout()
# save high-res PNG and SVG for vector/bitmap use
png_out = outputpath
svg_out = outputpath.replace('.png', '.svg')
plt.savefig(png_out, dpi=300, bbox_inches='tight')
try:
    plt.savefig(svg_out, format='svg', bbox_inches='tight')
except Exception:
    pass
print(f"3D pressure surface saved: {png_out} (and {svg_out} if supported)")

def plot_pressure_bar3d(grid_x, grid_y, times, pressure_grid, t_idx=None,
outputpath="q4_heatmap_bar3d.png"):
    """3D 柱状图：保留备用（当前主流程不生成）。"""

```

```

# Intentionally left as a utility for experimentation.
# (Not used in main flow per user request.)
return

def print_insights(grid_x, grid_y, times, demand_grid, supply_grid, reachable_grid,
pressure_grid):
    """打印关键洞察"""
    grid_size = len(grid_x)
    n_times = len(times)

    print("\n" + "=" * 70)
    print("空间-时间压力分析核心发现")
    print("=" * 70)

    # 最紧张的网格-时刻组合
    max_pressure = 0
    max_info = None
    for gx in range(grid_size):
        for gy in range(grid_size):
            for t_idx in range(n_times):
                p = pressure_grid[gx, gy, t_idx]
                if np.isfinite(p) and p > max_pressure:
                    max_pressure = p
                    max_info = (gx, gy, t_idx)

    if max_info:
        gx, gy, t_idx = max_info
        cx, cy = grid_x[gx], grid_y[gy]
        hour = times[t_idx]
        d = demand_grid[gx, gy, t_idx]
        s = reachable_grid[gx, gy, t_idx]
        print(f"\n► 最紧张网格：中心 ({cx:.1f}, {cy:.1f})")
        print(f" 🕒 时刻：{hour:.0f}:00")
        print(f" 🚗 需求：{d:.0f} 用户 | 可达供给：{s:.0f} 车位 | 压力比：{max_pressure:.2f}")

    # 整体统计
    total_demand_time = demand_grid.sum(axis=(0, 1))
    total_reachable_time = reachable_grid.sum(axis=(0, 1))
    total_supply_time = supply_grid.sum(axis=(0, 1))

    peak_demand_hour = times[np.argmax(total_demand_time)]
    valid_pressure = np.where(total_reachable_time > 0,
                              total_demand_time / total_reachable_time, 0)
    peak_pressure_hour = times[np.argmax(valid_pressure)]

    print(f"\n► 需求峰值时刻：{peak_demand_hour:.0f}:00 (最大活跃用户)")
    print(f"► 压力峰值时刻：{peak_pressure_hour:.0f}:00 (压力比最高)")

    # 供需失衡网格计数
    print(f"\n► 各时刻供需失衡网格（压力明细）:")
    header = f" {'时刻':<8} {'有需求网格':<12} {'可达供给>0':<12} {'高压(>2)':<12}"
    print(header)
    for t_idx, t in enumerate(times):
        print(f" {'-' * 56}")

```

```

p_slice = pressure_grid[:, :, t_idx]
d_slice = demand_grid[:, :, t_idx]

cells_with_demand = np.sum(d_slice > 0)
cells_with_supply = np.sum(reachable_grid[:, :, t_idx] > 0)

finite_p = np.isfinite(p_slice)
high_pressure = np.sum(finite_p & (p_slice > 2))
no_supply = np.sum((d_slice > 0) & (reachable_grid[:, :, t_idx] == 0))

print(f" {t:.0f}:00 {cells_with_demand:<12} {cells_with_supply:<12}
{high_pressure:<12} {no_supply:<12}")

# 空间聚集分析
print(f"\n► 需求最聚集区域（全天累计，前5）:")
total_demand_spatial = demand_grid.sum(axis=2)
flat_idx = np.argsort(-total_demand_spatial.flatten())[:5]
for idx in flat_idx:
    gx = idx % grid_size
    gy = idx // grid_size
    cx, cy = grid_x[gx], grid_y[gy]
    d_val = total_demand_spatial[gx, gy]
    print(f" 中心 ({cx:5.1f}, {cy:5.1f}) → 累计需求 {d_val:.0f} 用户")

print()

def main():
    print("=" * 70)
    print("空间-时间压力热力图分析")
    print("=" * 70)

    spots, slots, users = load_data()

    # 计算可达半径：使用用户 max_walk 的中位数
    radius = users["max_walk"].median()
    print(f"\n可达半径: {radius:.2f}m (用户最大步行距离中位数)")

    # 构建压力矩阵
    grid_size = 10
    print(f"网格: {grid_size}x{grid_size}")
    print(f"时间点: 8-22h (每小时)")

    grid_x, grid_y, times, demand_grid, supply_grid, reachable_grid, pressure_grid
= \
        build_spatial_temporal_pressure(spots, slots, users,
                                         grid_size=grid_size, radius=radius)

    # 打印洞察
    print_insights(grid_x, grid_y, times, demand_grid, supply_grid,
                   reachable_grid, pressure_grid)

    # 生成热力图
    plot_heatmap(spots, slots, users, grid_x, grid_y, times,
                 demand_grid, supply_grid, reachable_grid, pressure_grid)

    # plot_pressure_bar3d(grid_x, grid_y, times, pressure_grid,

```

```

outpath="q4_heatmap_bar3d.png")

# 生成汇总图
plot_demand_supply_summary(grid_x, grid_y, times,
                             demand_grid, supply_grid, reachable_grid,
                             pressure_grid)

print("分析完成! 输出文件:")
print("  - q4_heatmap.png (关键时刻空间压力热力图)")
print("  - q4_heatmap_summary.png (供需汇总分析)")

if __name__ == "__main__":
    main()

```

Q3 压力测试 `online_stress_test/`

`online_stress_test.py` — 在线算法压力测试

```

"""
在线算法压力测试
=====
测试A: 到达集中度扫描 - 将用户到达时间向午间压缩, 观察 EPR 变化
测试B: 步行阈值扰动 - w_j 乘以系数, 观察服务人数/平均距离/EPR 变化

输出:
    stress_test_A_concentration.csv
    stress_test_B_walk_threshold.csv

用法:
    python online_stress_test.py

依赖: pandas, numpy, scipy, pulp
随机种子: 42
"""

import sys
import os
import copy
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist

# — 路径设置: 优先使用 deepseek/utils.py 的数据加载逻辑 —————
REPO_ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), '..', '..'),
                                   '..')
sys.path.insert(0, os.path.join(REPO_ROOT, 'deepseek'))

try:

```

```

from utils import load_data, compute_distance_matrix
except ImportError:
    def load_data():
        """Fallback: 直接搜索 CSV"""
        from glob import glob
        for base in [REPO_ROOT, 'C:\\\\']:
            for p in glob(os.path.join(base, '**', 'spots.csv'), recursive=True):
                d = os.path.dirname(p)
                if os.path.exists(os.path.join(d, 'slots.csv')) and
os.path.exists(os.path.join(d, 'users.csv')):
                    def _read(path):
                        for enc in ['gbk', 'utf-8', 'utf-8-sig']:
                            try:
                                return pd.read_csv(path, encoding=enc)
                            except Exception:
                                continue
                        return pd.read_csv(path)
                    spots = _read(os.path.join(d, 'spots.csv'))
                    slots = _read(os.path.join(d, 'slots.csv'))
                    users = _read(os.path.join(d, 'users.csv'))
                    spots.columns = ['spot_id', 'x', 'y']
                    slots.columns = ['spot_id', 'start', 'end']
                    users.columns = ['user_id', 'dest_x', 'dest_y', 'max_walk',
'arrival', 'departure']
                    return spots, slots, users
                raise FileNotFoundError('找不到数据文件')

    def compute_distance_matrix(spots, users):
        return cdist(users[['dest_x', 'dest_y']].values, spots[['x', 'y']].values)

SEED = 42
OUT_DIR = os.path.dirname(os.path.abspath(__file__))

```

— 核心算法 —

```

def build_time_eligible(users_df, slots_df, dist_matrix, walk_scale=1.0):
    """构建时空可行边集 E^T, 返回 bool 矩阵 (n_users, n_spots)"""
    n_users, n_spots = dist_matrix.shape
    max_walk = users_df['max_walk'].values * walk_scale
    dist_ok = dist_matrix <= max_walk[:, np.newaxis] # (n_users, n_spots)

    time_ok = np.zeros((n_users, n_spots), dtype=bool)
    slot_groups = slots_df.groupby('spot_id')

    for j in range(n_users):
        a, d = users_df.iloc[j]['arrival'], users_df.iloc[j]['departure']
        for idx_i in np.where(dist_ok[j])[0]:
            spot_id = idx_i + 1
            if spot_id not in slot_groups.groups:
                continue
            for _, row in slot_groups.get_group(spot_id).iterrows():
                if row['start'] <= a and d <= row['end']:
                    time_ok[j, idx_i] = True
                    break
    return time_ok

```

```

def greedy_nearest(users_df, time_eligible, dist_matrix):
    """
    Greedy-Nearest 在线算法
    用户按 arrival 升序处理，为每位用户选最近的时间兼容且当前空闲的车位。
    返回 (served_count, avg_distance)
    """
    order = users_df['arrival'].argsort().values  # 按到达时间排序
    n_spots = dist_matrix.shape[1]
    # 每个车位的已分配区间列表 [(a, d), ...]
    spot_occupied = [[] for _ in range(n_spots)]

    served = 0
    total_dist = 0.0

    for j in order:
        a, d = users_df.iloc[j]['arrival'], users_df.iloc[j]['departure']
        best_spot = -1
        best_dist = np.inf

        for i in np.where(time_eligible[j])[0]:
            # 检查车位 i 在 [a, d) 是否空闲
            conflict = any(
                not (d <= oa or od <= a)
                for (oa, od) in spot_occupied[i]
            )
            if conflict:
                continue
            if dist_matrix[j, i] < best_dist:
                best_dist = dist_matrix[j, i]
                best_spot = i

            if best_spot >= 0:
                spot_occupied[best_spot].append((a, d))
                served += 1
                total_dist += best_dist

    avg_dist = total_dist / served if served > 0 else 0.0
    return served, avg_dist


def offline_milp(users_df, time_eligible, dist_matrix):
    """
    离线 MILP (Q2 模型)：两阶段字典序优化
    阶段1：最大化服务人数；阶段2：最小化总步行距离
    返回 (served_count, avg_distance)
    """
    try:
        import pulp
    except ImportError:
        print('[WARN] pulp 未安装，离线 MILP 跳过，返回 None')
        return None, None

    n_users, n_spots = time_eligible.shape
    # 构建冲突对
    conflict_pairs = {}  # spot_idx -> list of (j1, j2)
    for i in range(n_spots):

```

```

cands = np.where(time_eligible[:, i])[0]
pairs = []
for a_idx in range(len(cands)):
    for b_idx in range(a_idx + 1, len(cands)):
        j1, j2 = cands[a_idx], cands[b_idx]
        a1, d1 = users_df.iloc[j1]['arrival'], users_df.iloc[j1]
['departure']
        a2, d2 = users_df.iloc[j2]['arrival'], users_df.iloc[j2]
['departure']
        if a1 < d2 and a2 < d1: # 重叠
            pairs.append((j1, j2))
if pairs:
    conflict_pairs[i] = pairs

edges = [(j, i) for j in range(n_users) for i in range(n_spots) if
time_eligible[j, i]]
if not edges:
    return 0, 0.0

# 阶段1
prob1 = pulp.LpProblem('Q2_phase1', pulp.LpMaximize)
x = {(j, i): pulp.LpVariable(f'x_{j}_{i}', cat='Binary') for (j, i) in edges}
prob1 += pulp.lpSum(x[e] for e in edges)
for j in range(n_users):
    es = [(j, i) for (jj, i) in edges if jj == j]
    if es:
        prob1 += pulp.lpSum(x[e] for e in es) <= 1
for i in range(n_spots):
    es = [(j, i) for (j, ii) in edges if ii == i]
    if es:
        prob1 += pulp.lpSum(x[e] for e in es) <= 1
for i, pairs in conflict_pairs.items():
    for (j1, j2) in pairs:
        if (j1, i) in x and (j2, i) in x:
            prob1 += x[(j1, i)] + x[(j2, i)] <= 1

prob1.solve(pulp.PULP_CBC_CMD(msg=0))
N_star = int(round(pulp.value(prob1.objective)))

# 阶段2
prob2 = pulp.LpProblem('Q2_phase2', pulp.LpMinimize)
x2 = {(j, i): pulp.LpVariable(f'x2_{j}_{i}', cat='Binary') for (j, i) in edges}
prob2 += pulp.lpSum(dist_matrix[j, i] * x2[(j, i)] for (j, i) in edges)
for j in range(n_users):
    es = [(j, i) for (jj, i) in edges if jj == j]
    if es:
        prob2 += pulp.lpSum(x2[e] for e in es) <= 1
for i in range(n_spots):
    es = [(j, i) for (j, ii) in edges if ii == i]
    if es:
        prob2 += pulp.lpSum(x2[e] for e in es) <= 1
for i, pairs in conflict_pairs.items():
    for (j1, j2) in pairs:
        if (j1, i) in x2 and (j2, i) in x2:
            prob2 += x2[(j1, i)] + x2[(j2, i)] <= 1
prob2 += pulp.lpSum(x2[e] for e in edges) == N_star

```

```

prob2.solve(pulp.PULP_CBC_CMD(msg=0))

served = N_star
total_dist = sum(
    dist_matrix[j, i] * pulp.value(x2[(j, i)])
    for (j, i) in edges
    if pulp.value(x2[(j, i)]) is not None and pulp.value(x2[(j, i)]) > 0.5
)
avg_dist = total_dist / served if served > 0 else 0.0
return served, avg_dist

```

— 测试A: 到达集中度扫描 —

```

def compress_arrivals(users_df, kappa, center=12.5, rng=None):
    """
    将用户到达时间向 center（小时）压缩，集中度参数 kappa（小时）。
    压缩后到达时间 = center + (arrival - center) * kappa / original_half_range
    保持停车时长不变（departure = new_arrival + duration）。
    kappa 越小越集中；kappa = original_half_range 时不变。
    """
    users_new = users_df.copy()
    arrivals = users_df['arrival'].values
    durations = (users_df['departure'] - users_df['arrival']).values

    original_half_range = max(abs(arrivals - center).max(), 1e-6)
    scale = kappa / original_half_range
    new_arrivals = center + (arrivals - center) * scale
    # 确保 arrival >= 8.0, departure <= 22.0
    new_arrivals = np.clip(new_arrivals, 8.0, 22.0 - durations)
    new_departures = new_arrivals + durations
    new_departures = np.clip(new_departures, new_arrivals + 0.1, 22.0)

    users_new['arrival'] = new_arrivals
    users_new['departure'] = new_departures
    return users_new

def run_test_A(spots, slots, users, dist_matrix):
    """测试A: 集中度扫描"""
    print('=== 测试A: 到达集中度扫描 ===')
    arrivals = users['arrival'].values
    center = 12.5
    original_half_range = abs(arrivals - center).max()

    # kappa 从 original_half_range（原始）到 1h（高度集中）
    kappas = [original_half_range, original_half_range * 0.75,
               original_half_range * 0.5, 3.0, 2.0, 1.0]
    kappas = sorted(set(round(k, 2) for k in kappas), reverse=True)

    records = []
    for kappa in kappas:
        users_mod = compress_arrivals(users, kappa, center=center)
        te = build_time_eligible(users_mod, slots, dist_matrix)
        online_served, online_dist = greedy_nearest(users_mod, te, dist_matrix)
        offline_served, offline_dist = offline_milp(users_mod, te, dist_matrix)
        epr = online_served / offline_served if offline_served and offline_served >

```

```

0 else None
    rec = {
        'kappa_hours': round(kappa, 2),
        'online_served': online_served,
        'online_avg_dist_m': round(online_dist, 3),
        'offline_served': offline_served,
        'offline_avg_dist_m': round(offline_dist, 3) if offline_dist is not
None else None,
        'EPR': round(epr, 4) if epr is not None else None,
    }
    records.append(rec)
    print(f' kappa={kappa:.2f}h | online={online_served} | offline=
{offline_served} | EPR={epr}')

df = pd.DataFrame(records)
out_path = os.path.join(OUT_DIR, 'stress_test_A_concentration.csv')
df.to_csv(out_path, index=False, encoding='utf-8-sig')
print(f' → 已保存: {out_path}\n')
return df

```

— 测试B: 步行阈值扰动 —————

```

def run_test_B(spots, slots, users, dist_matrix):
    """测试B: 步行阈值扰动"""
    print('=== 测试B: 步行阈值扰动 ===')
    scales = [0.8, 0.9, 1.0, 1.1, 1.2]
    records = []

    for scale in scales:
        te = build_time_eligible(users, slots, dist_matrix, walk_scale=scale)
        online_served, online_dist = greedy_nearest(users, te, dist_matrix)
        offline_served, offline_dist = offline_milp(users, te, dist_matrix)
        epr = online_served / offline_served if offline_served and offline_served >
0 else None
        rec = {
            'walk_scale': scale,
            'online_served': online_served,
            'online_avg_dist_m': round(online_dist, 3),
            'offline_served': offline_served,
            'offline_avg_dist_m': round(offline_dist, 3) if offline_dist is not
None else None,
            'EPR': round(epr, 4) if epr is not None else None,
        }
        records.append(rec)
        print(f' scale={scale:.1f} | online={online_served} | offline=
{offline_served} | EPR={epr}')

df = pd.DataFrame(records)
out_path = os.path.join(OUT_DIR, 'stress_test_B_walk_threshold.csv')
df.to_csv(out_path, index=False, encoding='utf-8-sig')
print(f' → 已保存: {out_path}\n')
return df

```

— 主入口 —————

```

if __name__ == '__main__':
    np.random.seed(SEED)
    print('加载数据...')
    spots, slots, users = load_data()
    dist_matrix = compute_distance_matrix(spots, users)
    print(f'  spots={len(spots)}, slots={len(slots)}, users={len(users)}\n')

    df_A = run_test_A(spots, slots, users, dist_matrix)
    df_B = run_test_B(spots, slots, users, dist_matrix)

    print('全部测试完成。')
    print(f'  测试A结果: stress_test_A_concentration.csv')
    print(f'  测试B结果: stress_test_B_walk_threshold.csv')

```

plot_stress.py — EPR 曲线图生成

```

"""
压力测试结果可视化
=====
读取 online_stress_test.py 生成的 CSV，输出两张图：
    stress_test_A_epr_curve.png  - EPR vs 到达集中度
    stress_test_B_epr_curve.png  - EPR vs 步行阈值系数

用法：
    python plot_stress.py
"""

import os
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker

OUT_DIR = os.path.dirname(os.path.abspath(__file__))

plt.rcParams.update({
    'font.family': 'DejaVu Sans',
    'axes.unicode_minus': False,
    'figure.dpi': 150,
})

def plot_A():
    path = os.path.join(OUT_DIR, 'stress_test_A_concentration.csv')
    if not os.path.exists(path):
        print(f'[SKIP] 找不到 {path}，请先运行 online_stress_test.py')
        return

    df = pd.read_csv(path)

```

```

df = df.sort_values('kappa_hours')

fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))

# 左图: 服务人数
ax = axes[0]
ax.plot(df['kappa_hours'], df['online_served'], 'o-', label='Greedy-Nearest
(online)', color='steelblue')
ax.plot(df['kappa_hours'], df['offline_served'], 's--', label='Offline MILP
(Q2)', color='darkorange')
ax.set_xlabel('Arrival Concentration  $\kappa$  (hours)', fontsize=11)
ax.set_ylabel('Users Served', fontsize=11)
ax.set_title('(A) Served Users vs. Arrival Concentration', fontsize=11)
ax.legend(fontsize=9)
ax.grid(True, alpha=0.3)
ax.invert_xaxis() # 从宽松到集中 (左→右 kappa 减小)

# 右图: EPR
ax = axes[1]
ax.plot(df['kappa_hours'], df['EPR'], 'D-', color='crimson')
ax.axhline(0.632, color='gray', linestyle=':', linewidth=1.2, label='KVV lower
bound (0.632)')
ax.set_xlabel('Arrival Concentration  $\kappa$  (hours)', fontsize=11)
ax.set_ylabel('EPR (online / offline)', fontsize=11)
ax.set_title('(A) EPR vs. Arrival Concentration', fontsize=11)
ax.set_ylim(0, 1.05)
ax.yaxis.set_major_formatter(mticker.PercentFormatter(xmax=1, decimals=0))
ax.legend(fontsize=9)
ax.grid(True, alpha=0.3)
ax.invert_xaxis()

fig.tight_layout()
out = os.path.join(OUT_DIR, 'stress_test_A_epr_curve.png')
fig.savefig(out, bbox_inches='tight')
plt.close(fig)
print(f'已保存: {out}')

def plot_B():
    path = os.path.join(OUT_DIR, 'stress_test_B_walk_threshold.csv')
    if not os.path.exists(path):
        print(f'[SKIP] 找不到 {path}, 请先运行 online_stress_test.py')
        return

    df = pd.read_csv(path)
    df = df.sort_values('walk_scale')

    fig, axes = plt.subplots(1, 3, figsize=(14, 4.5))

    # 左图: 服务人数
    ax = axes[0]
    ax.plot(df['walk_scale'], df['online_served'], 'o-', label='Greedy-Nearest',
    color='steelblue')
    ax.plot(df['walk_scale'], df['offline_served'], 's--', label='Offline MILP',
    color='darkorange')
    ax.set_xlabel('Walk Threshold Scale', fontsize=11)
    ax.set_ylabel('Users Served', fontsize=11)

```

```

ax.set_title('(B) Served Users vs. Walk Scale', fontsize=11)
ax.legend(fontsize=9)
ax.grid(True, alpha=0.3)

# 中图：平均步行距离
ax = axes[1]
ax.plot(df['walk_scale'], df['online_avg_dist_m'], 'o-', label='Greedy-Nearest', color='steelblue')
ax.plot(df['walk_scale'], df['offline_avg_dist_m'], 's--', label='Offline MILP', color='darkorange')
ax.set_xlabel('Walk Threshold Scale', fontsize=11)
ax.set_ylabel('Avg Walk Distance (m)', fontsize=11)
ax.set_title('(B) Avg Distance vs. Walk Scale', fontsize=11)
ax.legend(fontsize=9)
ax.grid(True, alpha=0.3)

# 右图：EPR
ax = axes[2]
ax.plot(df['walk_scale'], df['EPR'], 'D-', color='crimson')
ax.axhline(0.632, color='gray', linestyle=':', linewidth=1.2, label='KVV lower bound')
ax.set_xlabel('Walk Threshold Scale', fontsize=11)
ax.set_ylabel('EPR (online / offline)', fontsize=11)
ax.set_title('(B) EPR vs. Walk Scale', fontsize=11)
ax.set_ylim(0, 1.05)
ax.yaxis.set_major_formatter(mticker.PercentFormatter(xmax=1, decimals=0))
ax.legend(fontsize=9)
ax.grid(True, alpha=0.3)

fig.tight_layout()
out = os.path.join(OUT_DIR, 'stress_test_B_epr_curve.png')
fig.savefig(out, bbox_inches='tight')
plt.close(fig)
print(f'已保存: {out}')

if __name__ == '__main__':
    plot_A()
    plot_B()
    print('绘图完成。')

```

假设鲁棒性检验 robustness/

robustness_test.py — 时间噪声与取消率扰动

```

"""
假设鲁棒性检验
=====

```

扰动A: 到达/离开时间噪声 – $\epsilon \sim \text{Uniform}[-10, 10]$ min, 重复20次

扰动B: 取消率 – 随机删除 $p\%$ 用户请求, $p \in \{5, 10, 15, 20\}$

输出:

robustness_A_time_noise.csv
robustness_B_cancellation.csv

用法:

python robustness_test.py

随机种子: 42

"""

```
import sys
import os
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist

REPO_ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), '..', '..', '..'))
sys.path.insert(0, os.path.join(REPO_ROOT, 'deepseek'))

try:
    from utils import load_data, compute_distance_matrix
except ImportError:
    def load_data():
        from glob import glob
        for base in [REPO_ROOT, 'C:\\']:
            for p in glob(os.path.join(base, '**', 'spots.csv'), recursive=True):
                d = os.path.dirname(p)
                if os.path.exists(os.path.join(d, 'slots.csv')) and
os.path.exists(os.path.join(d, 'users.csv')):
                    def _read(path):
                        for enc in ['gbk', 'utf-8', 'utf-8-sig']:
                            try:
                                return pd.read_csv(path, encoding=enc)
                            except Exception:
                                continue
                        return pd.read_csv(path)
                    spots = _read(os.path.join(d, 'spots.csv'))
                    slots = _read(os.path.join(d, 'slots.csv'))
                    users = _read(os.path.join(d, 'users.csv'))
                    spots.columns = ['spot_id', 'x', 'y']
                    slots.columns = ['spot_id', 'start', 'end']
                    users.columns = ['user_id', 'dest_x', 'dest_y', 'max_walk',
'arrival', 'departure']
                    return spots, slots, users
                    raise FileNotFoundError('找不到数据文件')

    def compute_distance_matrix(spots, users):
        return cdist(users[['dest_x', 'dest_y']].values, spots[['x', 'y']].values)

SEED = 42
OUT_DIR = os.path.dirname(os.path.abspath(__file__))
```

```

def build_time_eligible(users_df, slots_df, dist_matrix):
    n_users, n_spots = dist_matrix.shape
    max_walk = users_df['max_walk'].values
    dist_ok = dist_matrix <= max_walk[:, np.newaxis]
    time_ok = np.zeros((n_users, n_spots), dtype=bool)
    slot_groups = slots_df.groupby('spot_id')
    for j in range(n_users):
        a, d = users_df.iloc[j]['arrival'], users_df.iloc[j]['departure']
        for idx_i in np.where(dist_ok[j])[0]:
            spot_id = idx_i + 1
            if spot_id not in slot_groups.groups:
                continue
            for _, row in slot_groups.get_group(spot_id).iterrows():
                if row['start'] <= a and d <= row['end']:
                    time_ok[j, idx_i] = True
                    break
    return time_ok

def offline_milp(users_df, time_eligible, dist_matrix):
    try:
        import pulp
    except ImportError:
        return None, None

    n_users, n_spots = time_eligible.shape
    conflict_pairs = {}
    for i in range(n_spots):
        cand = np.where(time_eligible[:, i])[0]
        pairs = []
        for a_idx in range(len(cand)):
            for b_idx in range(a_idx + 1, len(cand)):
                j1, j2 = cand[a_idx], cand[b_idx]
                a1, d1 = users_df.iloc[j1]['arrival'], users_df.iloc[j1]
                a2, d2 = users_df.iloc[j2]['arrival'], users_df.iloc[j2]
                if a1 < d2 and a2 < d1:
                    pairs.append((j1, j2))
            if pairs:
                conflict_pairs[i] = pairs

    edges = [(j, i) for j in range(n_users) for i in range(n_spots) if
time_eligible[j, i]]
    if not edges:
        return 0, 0.0

    prob1 = pulp.LpProblem('rob_phase1', pulp.LpMaximize)
    x = {(j, i): pulp.LpVariable(f'x_{j}_{i}', cat='Binary') for (j, i) in edges}
    prob1 += pulp.lpSum(x[e] for e in edges)
    for j in range(n_users):
        es = [(j, i) for (jj, i) in edges if jj == j]
        if es:
            prob1 += pulp.lpSum(x[e] for e in es) <= 1
    for i in range(n_spots):

```

```

        es = [(j, i) for (j, ii) in edges if ii == i]
        if es:
            prob1 += pulp.lpSum(x[e] for e in es) <= 1
    for i, pairs in conflict_pairs.items():
        for (j1, j2) in pairs:
            if (j1, i) in x and (j2, i) in x:
                prob1 += x[(j1, i)] + x[(j2, i)] <= 1
    prob1.solve(pulp.PULP_CBC_CMD(msg=0))
    N_star = int(round(pulp.value(prob1.objective)))

    prob2 = pulp.LpProblem('rob_phase2', pulp.LpMinimize)
    x2 = {(j, i): pulp.LpVariable(f'x2_{j}_{i}', cat='Binary') for (j, i) in edges}
    prob2 += pulp.lpSum(dist_matrix[j, i] * x2[(j, i)] for (j, i) in edges)
    for j in range(n_users):
        es = [(j, i) for (jj, i) in edges if jj == j]
        if es:
            prob2 += pulp.lpSum(x2[e] for e in es) <= 1
    for i in range(n_spots):
        es = [(j, i) for (j, ii) in edges if ii == i]
        if es:
            prob2 += pulp.lpSum(x2[e] for e in es) <= 1
    for i, pairs in conflict_pairs.items():
        for (j1, j2) in pairs:
            if (j1, i) in x2 and (j2, i) in x2:
                prob2 += x2[(j1, i)] + x2[(j2, i)] <= 1
    prob2 += pulp.lpSum(x2[e] for e in edges) == N_star
    prob2.solve(pulp.PULP_CBC_CMD(msg=0))

    total_dist = sum(
        dist_matrix[j, i] * pulp.value(x2[(j, i)])
        for (j, i) in edges
        if pulp.value(x2[(j, i)]) is not None and pulp.value(x2[(j, i)]) > 0.5
    )
    avg_dist = total_dist / N_star if N_star > 0 else 0.0
    return N_star, avg_dist

```

— 扰动A: 时间噪声 —

```

def run_perturbation_A(spots, slots, users, dist_matrix, n_trials=20,
noise_min_h=-10/60, noise_max_h=10/60):
    """
    到达/离开时间加均匀噪声，重复 n_trials 次，记录均值±标准差
    noise 单位: 小时 (10 min = 10/60 h)
    """
    print('=== 扰动A: 时间噪声 ===')
    rng = np.random.default_rng(SEED)

    baseline_te = build_time_eligible(users, slots, dist_matrix)
    baseline_served, baseline_dist = offline_milp(users, baseline_te, dist_matrix)
    print(f'  基准（无扰动）: served={baseline_served}, avg_dist=
{baseline_dist:.3f}m')

    served_list, dist_list = [], []
    for trial in range(n_trials):
        users_mod = users.copy()
        eps_a = rng.uniform(noise_min_h, noise_max_h, size=len(users))

```

```

eps_d = rng.uniform(noise_min_h, noise_max_h, size=len(users))
durations = (users['departure'] - users['arrival']).values
new_a = np.clip(users['arrival'].values + eps_a, 8.0, 22.0)
new_d = np.clip(new_a + durations + eps_d, new_a + 0.05, 22.0)
users_mod['arrival'] = new_a
users_mod['departure'] = new_d

te = build_time_eligible(users_mod, slots, dist_matrix)
served, avg_dist = offline_milp(users_mod, te, dist_matrix)
if served is not None:
    served_list.append(served)
    dist_list.append(avg_dist)
print(f' trial {trial+1:2d}: served={served}, avg_dist={avg_dist:.3f}m')

records = [{
    'noise_range_min': noise_min_h * 60,
    'noise_range_max': noise_max_h * 60,
    'n_trials': n_trials,
    'baseline_served': baseline_served,
    'baseline_avg_dist_m': round(baseline_dist, 3) if baseline_dist else None,
    'mean_served': round(np.mean(served_list), 2),
    'std_served': round(np.std(served_list), 2),
    'mean_avg_dist_m': round(np.mean(dist_list), 3),
    'std_avg_dist_m': round(np.std(dist_list), 3),
    'served_change_pct': round((np.mean(served_list) - baseline_served) /
baseline_served * 100, 2) if baseline_served else None,
}]
df = pd.DataFrame(records)
out_path = os.path.join(OUT_DIR, 'robustness_A_time_noise.csv')
df.to_csv(out_path, index=False, encoding='utf-8-sig')
print(f' → 已保存: {out_path}\n')
return df

```

— 扰动B: 取消率 —

```

def run_perturbation_B(spots, slots, users, dist_matrix, cancel_rates=None,
n_trials=10):
    """
    随机删除 p% 用户请求, 重复 n_trials 次, 记录均值±标准差
    """
    if cancel_rates is None:
        cancel_rates = [0.05, 0.10, 0.15, 0.20]
    print('=== 扰动B: 取消率 ===')
    rng = np.random.default_rng(SEED)

    baseline_te = build_time_eligible(users, slots, dist_matrix)
    baseline_served, baseline_dist = offline_milp(users, baseline_te, dist_matrix)
    print(f' 基准（无取消）: served={baseline_served}')

    records = []
    for p in cancel_rates:
        served_list, dist_list = [], []
        for trial in range(n_trials):
            n_cancel = int(len(users) * p)
            cancel_idx = rng.choice(len(users), size=n_cancel, replace=False)
            keep_mask = np.ones(len(users), dtype=bool)

```

```

keep_mask[cancel_idx] = False
users_mod = users[keep_mask].reset_index(drop=True)
dist_mod = dist_matrix[keep_mask]

te = build_time_eligible(users_mod, slots, dist_mod)
served, avg_dist = offline_milp(users_mod, te, dist_mod)
if served is not None:
    served_list.append(served)
    dist_list.append(avg_dist)

rec = {
    'cancel_rate_pct': int(p * 100),
    'n_trials': n_trials,
    'baseline_served': baseline_served,
    'mean_served': round(np.mean(served_list), 2),
    'std_served': round(np.std(served_list), 2),
    'mean_avg_dist_m': round(np.mean(dist_list), 3),
    'std_avg_dist_m': round(np.std(dist_list), 3),
    'served_change_pct': round((np.mean(served_list) - baseline_served) /
baseline_served * 100, 2),
    }
    records.append(rec)
    print(f' p={int(p*100)}%: mean_served={rec["mean_served"]}±
{rec["std_served"]}, change={rec["served_change_pct"]}%' )

df = pd.DataFrame(records)
out_path = os.path.join(OUT_DIR, 'robustness_B_cancellation.csv')
df.to_csv(out_path, index=False, encoding='utf-8-sig')
print(f' → 已保存: {out_path}\n')
return df

# — 主入口 —————

if __name__ == '__main__':
    np.random.seed(SEED)
    print('加载数据...')
    spots, slots, users = load_data()
    dist_matrix = compute_distance_matrix(spots, users)
    print(f' spots={len(spots)}, slots={len(slots)}, users={len(users)}\n')

    df_A = run_perturbation_A(spots, slots, users, dist_matrix, n_trials=20)
    df_B = run_perturbation_B(spots, slots, users, dist_matrix, n_trials=10)

    print('全部鲁棒性检验完成。')

```